Centre universitari adscrit a la

Universitat Pompeu Fabra Barcelona

Bachelor in Computer Engineering — Management and Information Systems

Ethical Pentesting in Virtualized Environments with EVE-NG

Memory

Volume I

Eloi Egea Rada
Supervisor: Pere Vidiella i Catalan

April 2026

Contents

List of Figures V

List of Tables VII

Glossary IX

Dedication and Acknowledgements 1

1 Introduction 3
1.1 Context 4
1.2 Proposed Solution 5

2 State of the Art 7
2.1 Existing Platforms Overview 7
2.2 Security Frameworks and Methodologies 8
2.2.1 OWASP Top 10 8
2.2.2 CIS Critical Controls 8
2.2.3 NIST Cybersecurity Framework 8
2.3 Prior Academic Research 9
2.4 Gap Analysis: What Existing Solutions Miss 9
2.5 The Need for EVE-NG Based Labs 10

3 Feasibility Study 13
3.1 Gantt Chart and Task Definition 13
3.1.1 Critical Path 13
3.2 Budget 14
3.2.1 Human Resources 14
3.2.2 Hardware and Software 15
3.2.3 Total Budget 15
3.3 Feasibility Analysis 15
3.3.1 Technical Feasibility 15
3.3.2 Economic Feasibility 17
3.3.3 Environmental Feasibility 17
3.3.4 Legal Aspects 18
3.3.5 Gender and Diversity Perspective 19

4 Objectives and Scope 21
4.1 Specific Objectives (Measurable with KPIs) 21
4.1.1 OE1: Design and Implement EVE-NG Labs 21
4.1.2 OE2: Develop Automation Scripts 21
4.1.3 OE3: Develop Structured Teaching Material 22
4.1.4 OE4: Validate Usability and Educational Effectiveness 22
4.2 Definition of the Client and Final User 23
4.2.1 Principal Client 23

4.2.2 Primary End Users 23
4.3 Scope Boundaries 23
4.4 KPIs and Performance Indicators 25

5 Methodology 27
5.1 Research and Development Phases 27
5.1.1 Phase 0: Preparation and Infrastructure (July–November 2025) 27
5.1.2 Phase 1: Preliminary Project Proposal and Conceptual Design (October 2025–January 2026) 27
5.1.3 Phase 2: Implementation and Interim Validation (January–March 2026) 28
5.1.4 Phase 3: Completion and Final Validation (March–May 2026) 28
5.2 Information Search Strategies 28
5.3 Development Methodology 29
5.4 Platform Selection Rationale 29
5.5 Validation and Success Criteria 29
5.6 Risk Management 30

6 Functional and Non-Functional Requirements 31
6.1 Functional Requirements 31
6.1.1 RF1: Design and Implement EVE-NG Topologies . . . 31
6.1.2 RF2: Implement Vulnerabilities and Attack Scenarios 31
6.1.3 RF3: Create Structured Assessment Materials 32
6.2 Technological Requirements 33
6.2.1 TR1: Virtualisation Infrastructure 33
6.2.2 TR2: Security Tools and Penetration Testing Framework 33
6.2.3 TR3: Scripting and Automation 33
6.3 Non-Functional Requirements 34
6.4 Requirements Traceability Matrix 34

7 Development 37
7.1 Platform Setup 37
7.2 Image Management 38
7.2.1 Image Selection 38
7.2.2 Image Organisation in EVE-NG 39
7.2.3 ISO Upload Automation 39
7.3 EVE-NG Lab Management Interface 40
7.4 Infrastructure Problems 41
7.4.1 EVE-NG on VMware 41
7.4.2 Node Configuration Persistence 41
7.4.3 LVM Volume Group Naming Conflict 42
7.4.4 Memory Capacity Constraint 42
7.5 Lab Topologies 43
7.6 Lab 1: Reconnaissance and Enumeration 43
7.6.1 Problems Encountered and Solutions 44

7.7 Lab 2: Web Application Vulnerabilities — SYNAPSE Intelligence Portal 46
7.7.1 Problems Encountered and Solutions 47
7.8 Lab 3: Incident Response and Log Forensics — HELIX Systems 49
7.8.1 Problems Encountered and Solutions 50

Bibliography **53**

A EVE-NG Installation on Proxmox VE **59**

B Lab 1: PEBCAK Corp — Instructor Reference **61**

C Lab 2: SYNAPSE Intelligence Portal — Instructor Reference **65**

D Lab 3: HELIX Systems — Instructor Reference **69**

E Cybersecurity Training Platform Descriptions **71**
E.1 HackTheBox 71
E.2 TryHackMe 71
E.3 Cisco Networking Academy 72
E.4 SEED Labs 72
E.5 Game of Active Directory (GOAD) 72
E.6 VulnHub 73
E.7 Hack4u Academy 73

F Objectives: Deliverables, Detailed KPIs, and Audience **75**
F.1 Deliverables by Specific Objective 75
F.1.1 OE1: Design and Implement EVE-NG Labs 75
F.1.2 OE2: Develop Automation Scripts 75
F.1.3 OE3: Develop Structured Teaching Material 75
F.1.4 OE4: Validate Usability and Educational Effectiveness 76
F.2 Detailed KPI Tables 76
F.2.1 Technical Indicators 76
F.2.2 Educational Indicators 77
F.2.3 Quality Indicators 77
F.3 Extended Audience Analysis 77
F.3.1 Secondary End Users 77
F.3.2 Potential Audience Beyond Tecnocampus 77

G Methodology: Detailed Phase Activities and Quality Assurance **79**
G.1 Detailed Phase Activity Lists 79
G.1.1 Phase 0: Preparation and Infrastructure (July–November 2025) 79
G.1.2 Phase 1: Preliminary Project Proposal and Conceptual Design (October 2025–January 2026) 79
G.1.3 Phase 2: Implementation and Interim Validation (January–March 2026) 81

G.1.4 Phase 3: Completion and Final Validation (March–May 2026) 82
G.2 Quality Assurance Procedures 83
G.2.1 Technical Validation 83
G.2.2 Pedagogical Validation 83
G.2.3 Usability Testing 83
G.3 Documentation and Version Control 84

H Requirements: Detailed Sub-Requirements 85
H.1 Functional Requirements: Sub-Specifications 85
H.1.1 RF1: Design and Implement EVE-NG Topologies . . . 85
H.1.2 RF2: Implement Vulnerabilities and Attack Scenarios 86
H.1.3 RF3: Create Structured Assessment Materials 86
H.2 Technological Requirements: Sub-Specifications 86
H.2.1 TR1: Virtualisation Infrastructure 86
H.2.2 TR2: Security Tools and Penetration Testing Framework 87
H.2.3 TR3: Scripting and Automation 87
H.3 Non-Functional Requirements: Detailed Subsections 87
H.3.1 NFR1: Performance and Efficiency 87
H.3.2 NFR2: Compliance with Security Frameworks 88
H.3.3 NFR3: Reliability and Availability 89
H.3.4 NFR4: Security and Ethical Compliance 89
H.3.5 NFR5: Usability and Support 89
H.3.6 NFR6: Institutional Sustainability 90

I Feasibility: Task Breakdown and NIST Risk Assessment 91
I.1 Phase Task Lists 91
I.1.1 Phase 0: Preparation (July–November 2025) 91
I.1.2 Phase 1: Preliminary Project Proposal (October 2025–January 2026) 91
I.1.3 Phase 2: Interim Report (January–March 2026) 92
I.1.4 Phase 3: Final Delivery (March–May 2026) 93
I.2 NIST SP 800-30 Five-Phase Risk Assessment 93
I.2.1 Phase 1: Asset Identification and Scope Definition . 94
I.2.2 Phase 2: Threat and Vulnerability Identification . . . 95
I.2.3 Phase 3: Impact and Likelihood Analysis 95
I.2.4 Phase 4: Risk Evaluation and Prioritisation 96
I.2.5 Phase 5: Mitigation Strategies 96
I.3 Detailed Budget Tables 99
I.3.1 Software and Licences 99
I.3.2 Other Operational Costs 99

List of Figures

7.1 EVE-NG File Manager showing the lab directory structure, import controls, and topology thumbnails. 40

List of Tables

2.1 Comparative Analysis: Existing Cybersecurity Training Platforms 7

3.1 Distribution of tasks by category 13
3.2 Critical path tasks 14
3.3 Human resources cost (junior engineer rate, 15 €/h, theoretical) 14
3.4 Hardware and software costs (all software is free/open-source) 15
3.5 Complete project budget 15
3.6 Required technical competencies 16
3.7 Risk assessment 16
3.8 Cost comparison 17
3.9 CO^2 comparison 17

4.1 Key performance indicators for the lab teaching package 25

5.1 Key project risks and mitigation strategies 30

6.1 Non-functional requirements summary 34
6.2 Requirements traceability 34

7.1 Overview of the four EVE-NG lab topologies designed for the course. 43
7.2 Lab 1 IP addressing scheme 44
7.3 Lab 1 techniques and framework mapping 44
7.4 Lab 1 flags and acquisition techniques 46
7.5 Lab 2 IP addressing scheme 47
7.6 Lab 2 OWASP Top 10 vulnerability chain 47
7.7 Lab 3 IP addressing scheme 49
7.8 Lab 3 MITRE ATT&CK technique coverage 50

A.1 EVE-NG VM configuration parameters 59

B.1 Lab 1 credentials (instructor reference — do not distribute to students) 61
B.2 Lab 1 network segments 61
B.3 Lab 1 expected student workflow 62
B.4 Lab 1 flags 63

C.1 Lab 2 credentials (instructor reference — do not distribute to students) 65
C.2 Lab 2 vulnerability chain and flag locations 66

D.1 Lab 3 accounts (instructor reference) 69
D.2 Lab 3 pre-staged attack timeline 69

D.3 Lab 3 evidence artefact locations 70

F.1 Technical KPIs for EVE-NG lab deployment 76
F.2 Educational KPIs for lab effectiveness 77
F.3 Quality KPIs for lab package 77

I.1 Risk assessment matrix with probability and impact (NIST SP 800-30) 96
I.2 Software and licence costs (all free/open-source for educational use) 99
I.3 Detailed breakdown of additional operational costs 99

Glossary

API REST

Application Programming Interface following the *Representational State Transfer* (REST) architectural style, using HTTP to enable communication between systems and supporting standard resource operations.

Burp Suite

An integrated web application security testing platform enabling the interception, modification, and analysis of HTTP/HTTPS traffic. Used for web vulnerability exercises in Lab 2.

CIS Critical Controls

A consensus-based set of 18 prioritised security controls from the Center for Internet Security, representing best practices for defending against the most prevalent attack vectors.

Cross-Site Scripting (XSS)

A vulnerability enabling the injection of malicious client-side scripts into web pages viewed by other users, thereby compromising session tokens or exfiltrating sensitive data.

CTF (Capture The Flag)

A cybersecurity competition format in which participants solve challenges to find hidden strings (flags) that prove successful exploitation. The lab progression model used throughout this thesis is based on a linear CTF chain.

CVSS (Common Vulnerability Scoring System)

A standardised framework for quantifying vulnerability severity through a numerical score—ranging from 0 to 10—that reflects exploitability and potential impact on affected systems.

DMZ (Demilitarized Zone)

A neutral security segment segregating an organisation's internal network from the public Internet, hosting externally-accessible

services whilst protecting critical internal resources from direct exposure.

DNS (Domain Name System)

The hierarchical distributed naming system that resolves human-readable hostnames to IP addresses. Used in Lab 1 as the entry point for reconnaissance: the custom hostname `lab1` is resolved via the pfSense Unbound DNS resolver.

Enumeration

A methodical process of systematically identifying and documenting network resources, active services, user accounts, and group memberships—typically following network scanning. Essential for pinpointing viable attack targets.

Ethical Hacking

The authorised and legal practice of proactively exploiting security vulnerabilities with explicit permission, conducted within a framework of ethical and legal responsibility. Conceptual foundation of this educational endeavour.

EVE-NG

Emulated Virtual Environment — Next Generation; a network emulation platform enabling the construction of complex multi-node topologies with diverse virtual machines and networking devices. Foundational platform of this thesis.

Exploit

A piece of code, technique, or procedure that strategically targets a specific vulnerability to achieve unauthorised access or execute arbitrary code on a target system.

HomeLab

A personal infrastructure environment combining server hardware and virtualisation software for educational purposes, experimentation, and professional development. Serves as the physical foundation for this thesis.

Incident Response

A structured methodology for detecting, containing, eradicating, and recovering from cybersecurity incidents. Lab 3 of this thesis simulates an active incident at HELIX Systems, requiring students to follow an IR workflow aligned with NIST SP 800-61.

Kali Linux

A Debian-based Linux distribution tailored for penetration testing and security auditing, providing hundreds of pre-installed tools for scanning, exploitation, and analysis. Widely used in industry and referenced in certification curricula such as OSCP.

Lab Topology

The architectural arrangement of virtual machines, network segments, and services within an educational laboratory environment, designed to simulate realistic penetration testing scenarios.

Lateral Movement

A post-exploitation technique in which an attacker uses a compromised system as a pivot point to access other systems within the same network, typically with the aim of reaching higher-value targets or sensitive data. Demonstrated in Lab 3 via SSH credential reuse from Server-Web to Server-DB.

Local Privilege Escalation

A privilege escalation technique executed locally on an already-compromised machine, exploiting vulnerabilities in the operating system kernel or misconfigured service permissions.

Metasploit Framework

An open-source exploitation framework facilitating the development, customisation, and execution of exploits against target systems. Industry standard for professional penetration testing engagements.

NAT (Network Address Translation)

A technique in which a router or firewall modifies the source or destination IP addresses of packets in transit, enabling multiple

hosts on a private network to share a single public address and allowing controlled exposure of internal services.

Network Scanning

The process of discovering active hosts, open ports, and running services across a network using specialised tools such as Nmap. The foundational step of offensive reconnaissance.

Network Topology

The physical and logical arrangement of network components—including computers, servers, networking devices, and interconnections. Critical for designing realistic laboratory environments.

nginx

A high-performance open-source web server and reverse proxy. Used in Lab 1 as the HTTP server on the target node, serving the PEBCAK Corp portal.

NIST Cybersecurity Framework

A standards-based framework from the National Institute of Standards and Technology for managing cybersecurity risk, organised into five functional domains: Identify, Protect, Detect, Respond, and Recover.

Nmap

An open-source network scanning tool that discovers active hosts, enumerates open ports, identifies running services, and fingerprints operating systems. Essential during the reconnaissance phase.

OWASP

Open Web Application Security Project; an international non-profit community dedicated to advancing web application security. Publishes the industry-standard Top 10 catalogue of critical vulnerabilities and comprehensive testing methodologies.

OWASP ZAP

An open-source web application scanner offering both automated scanning and interactive testing workflows. A freely available alternative to commercial web security tools.

Packet Capture

The process of intercepting and recording data packets traversing a network, enabling the analysis of protocols, unencrypted communications, and anomalous traffic patterns.

Parrot OS

A Debian-based Linux distribution oriented towards security testing, privacy, and development. Used as the operating system of the attacker node in the lab environment of this thesis.

Payload

The component or data transmitted as part of an attack vector, encapsulating the malicious code or instructions the attacker intends to execute within the compromised system.

Pentesting

Penetration testing; an authorised, controlled simulation of cyberattacks designed to identify security vulnerabilities across systems, networks, and applications. Fundamental component of this thesis.

pfSense

An open-source firewall and router platform based on FreeBSD, distributed by Netgate. Used in this project as the perimeter firewall, providing NAT, port forwarding, and DNS resolution for the lab environment.

Privilege Escalation

A technique enabling a user with limited permissions to obtain elevated privileges—such as administrator or root access—thereby executing previously restricted operations. Central focus of Lab 4.

Proxmox VE

An open-source server virtualisation platform based on Debian Linux, supporting both KVM virtual machines and LXC containers. Used in this project as the hypervisor hosting the EVE-NG virtual machine.

Reconnaissance

The initial phase of a penetration test during which the attacker gathers intelligence about the target—encompassing systems, users, networks, and services—typically through passive techniques without direct system access. Primary focus of Lab 1 of this thesis.

Root Access

Maximum-level administrative access on Unix/Linux systems, equivalent to Administrator privileges on Windows. A typical objective in attack chains and the culmination of exploitation phases.

SQL Injection

A critical vulnerability that permits an attacker to insert malicious SQL code into database queries, enabling unauthorised data access, modification, or deletion. Primary focus of Lab 2.

SSH (Secure Shell)

A cryptographic network protocol providing secure command-line access, file transfer, and tunnelling over an unsecured network. Used in Lab 1 as the final step of the student workflow to access the target server and retrieve the flag.

TFG (Bachelor's Thesis)

A mandatory capstone project in Spanish university degree programmes, demonstrating the comprehensive knowledge and competencies acquired throughout the academic programme.

Virtual Machine

A software-based emulation of a complete computer system running within a physical host, enabling the concurrent execution of multiple operating systems and the creation of isolated, reproducible laboratory environments.

Vulnerability

A flaw or weakness inherent in a system, application, or infrastructure that can be exploited by an attacker to gain unautho-

rised access, inflict damage, or compromise data integrity and confidentiality.

Vulnerability Assessment

A systematic process of identifying, quantifying, and prioritising security vulnerabilities across systems, networks, and applications. Typically precedes comprehensive penetration testing engagements.

VyOS

An open-source network operating system based on Debian, providing routing, firewalling, and VPN functionality. Used in this project as the internal Layer 3 router connecting the lab network segments.

Wireshark

An open-source network protocol analyser that captures and examines network traffic in real time. Particularly relevant for network traffic analysis exercises and referenced in Lab 2 and Lab 3 of this thesis.

Dedication and Acknowledgements

Dedication

This work is dedicated to all students and professionals interested in cybersecurity, particularly in ethical pentesting. We hope that the labs and scenarios presented in this bachelor's thesis will be useful for their training and professional development.

Acknowledgements

Institutional Acknowledgements

I would like to express my sincere gratitude to my tutor, Pere Vidiella i Catalan, for his guidance, trust, and constructive feedback throughout this project. I am also deeply grateful to the faculty and staff of the Computer Engineering program for providing the academic framework, resources, and supportive environment that made this work possible. Their dedication to excellence in teaching and mentoring has been instrumental in shaping both the technical scope and the academic rigor of this thesis.

Personal Acknowledgements

This thesis is dedicated, above all, to my grandmother, who passed away just before I began my degree—shortly after my university admission and the day before my English examination. Choosing to write this thesis in English is, in part, a meaningful way to close a chapter that marked the beginning of my university journey without the person who meant the most to me and who, in many ways, was like a mother to me.

I am also profoundly grateful to my friends, my partner, and my partner's family for their unwavering support and encouragement, especially during the most difficult moments of my studies, including a

period when I faced a serious health challenge that could have left me paralyzed. Their presence, care, and belief in me have been essential to completing this work.

Use of Artificial Intelligence Tools

During the development of this thesis, complementary AI tools were used exclusively for review and error-detection purposes: a locally-hosted language model (LM Studio) and an online assistant (Claude, Anthropic). These tools were not used to generate any academic content; all analysis, design, implementation, and writing are the author's own work.

Personal Reflection

The completion of this Bachelor's Thesis has been an enriching experience that has allowed me to deepen my knowledge of cybersecurity and develop technical and project management skills. I have learned the importance of practice in learning this discipline and how virtualized environments can facilitate this process. This project has motivated me to continue exploring the field of cybersecurity and to contribute to the training of future professionals in this critical area.

1 Introduction

Cybersecurity education faces a critical challenge: the disconnect between theoretical knowledge of IT systems and practical understanding of how security failures manifest in integrated environments. While students in the Computer Engineering degree program learn fundamental concepts in databases, network protocols, operating system internals, and software architecture, they rarely experience how misconfigurations and design flaws in these domains directly translate to security vulnerabilities.

Current training approaches amplify this gap. Commercial platforms such as HackTheBox[1] and TryHackMe[2] focus on isolated challenge-based scenarios that abstract away system complexity. Professional pentesting certifications (e.g., CEH, OSCP[3], [4]) expect hands-on experience in realistic environments, yet many academic programmes struggle to provide such environments due to cost, complexity, or vendor lock-in.

Within the TecnoCampus Bachelor's Degree in Computer Engineering (Management and Information Systems track), the "Introduction to Cybersecurity" course currently lacks a structured, reusable, and automatable lab environment that:

- Aligns pentesting scenarios with specific degree learning outcomes
- Enables students to recognise vulnerabilities within integrated systems (not isolated exploits)
- Provides reproducible environments for consistent assessment
- Reduces instructor workload through automation and documentation
- Remains cost-effective and institution-independent (open-source, locally maintained)

1.1 Context

The Centre Universitari TecnoCampus [5] offers applied science degrees oriented towards professional practice in technology, business, creative industries, and health. Affiliated with the Universitat Pompeu Fabra, it carries that university's quality seal. Established in 2023 through the merger of three schools—the Escola Superior Politecnica (ESUPT), the Escola Superior de Ciencies Socials i de l'Empresa (ESCSET), and the Escola Superior de Ciencies de la Salut (ESCST)—it is located at the technology park of the same name in Mataro, combining academic infrastructure with an entrepreneurial ecosystem that facilitates student participation in innovation and industry-facing projects.

The primary beneficiary of this project is the Bachelor's Degree in Computer Engineering (Management and Information Systems track). Students in this programme acquire foundational competencies in networks, operating systems, databases, and software engineering all of which are directly relevant to security practice. The *Introduction to Cybersecurity* course is a natural integration point for these skills, yet it currently lacks a dedicated hands-on environment.

Three perspectives illustrate the gap this project addresses:

Institutional perspective. The absence of dedicated cybersecurity lab infrastructure limits the institution's capacity to offer competitive, hands-on security training aligned with degree learning outcomes. Commercial alternatives such as HackTheBox and TryHackMe are externally hosted, non-customisable, and not designed to map to specific academic curricula or assessment rubrics.

Student perspective. Students encounter security concepts as isolated theoretical constructs rather than as failures observable in realistic, integrated environments. Without exposure to multi-component systems—firewalls, routers, servers, and applications working together—they cannot develop the diagnostic reasoning required in professional security roles.

Instructor perspective. Designing and maintaining practical security exercises is resource-intensive. The absence of reusable, automatable lab environments forces instructors to improvise or rely on third-party platforms, reducing pedagogical control and limiting reproducibility across cohorts.

1.2 Proposed Solution

This project proposes a reusable penetration testing lab package built on EVE-NG [6] and deployed on self-hosted infrastructure, designed specifically for the *Introduction to Cybersecurity* course at TecnoCampus. The package comprises four progressive lab scenarios covering network reconnaissance and enumeration, web application vulnerabilities, incident response and log forensics, and privilege escalation. Labs 1–3 are fully implemented and validated; Lab 4 (privilege escalation) is in progress and will be completed in Phase 3. Each lab is self-contained: it includes an EVE-NG topology file (`.unl`), a student lab guide and an instructor resolution guide (instructor resolution guide), an assessment rubric, and supporting configuration and automation scripts. The entire package is open-source, vendor- independent, and deployable on any institution running a compatible hypervisor. Instructor deployment references, credentials, and lab-specific configuration details are compiled in the Annexos (Volume II), which accompanies this memory and is also published online at `docs/web/docs/assets/official\_Documents/annexos.md`.

2 State of the Art

Cybersecurity education has experienced exponential growth over the past decade, driven by increasing industry demand for skilled professionals and the accessibility of online learning platforms. While students have unprecedented access to training resources, the fragmentation of educational approaches has created gaps between abstract theoretical knowledge and practical, integrated system security. This chapter examines existing platforms and methodologies, identifies gaps in current offerings, and positions this bachelor's thesis within the broader context of cybersecurity pedagogical innovation.

2.1 Existing Platforms Overview

The major platforms used for cybersecurity training share a common limitation: they are designed for independent learners or certification preparation rather than curriculum-aligned institutional deployment. Table 2.1 summarises their key characteristics. Platforms such as HackTheBox [1] and TryHackMe [2] are externally hosted and cannot be deployed on institutional infrastructure. SEED Labs [7] and GOAD [8] support local deployment but cover only isolated concepts or narrow domains. VulnHub [9] and Hack4u [10] lack automation and assessment integration. Cisco Networking Academy [11] provides formal structure but is tightly coupled to vendor technology and limited to network simulation.

Platform	Approach	Deploy	Curriculum	Auto	Network
HackTheBox	CTF	No	Minimal	No	Isolated
TryHackMe	Gamified	No	Structured	No	Limited
Cisco Academy	Academic	Yes (PT)	Strong	Limited	Net-focused
SEED Labs	Conceptual	Yes (Docker)	Concept-based	Minimal	Isolated
GOAD	AD-focused	Yes (Terraform)	None	Moderate	AD Domain
VulnHub	Community	Yes	None	No	Isolated
Hack4u Academy	Methodology	External	Strong	Minimal	Limited

Table 2.1: Comparative Analysis: Existing Cybersecurity Training Platforms

Full individual platform descriptions, including detailed discussion of

each platform's pedagogical approach, deployment model, and specific limitations, are provided in Appendix E.

2.2 Security Frameworks and Methodologies

Several authoritative frameworks inform cybersecurity education and guide both attack methodologies and defensive best practices.

2.2.1 OWASP Top 10

The OWASP Top 10 is a regularly updated list of the most critical web application security vulnerabilities [12]. For educational purposes it provides a focused, industry-recognised curriculum that prioritises the attack vectors students are most likely to encounter in practice. Lab 2 of this project is directly aligned with OWASP Top 10 categories.

2.2.2 CIS Critical Controls

The CIS Critical Controls (v8.0) represent a prioritised set of defensive techniques spanning network defence, data protection, and incident response [13]. Labs 1 and 3 incorporate defensive controls so that students understand not only how to attack but why specific security measures are implemented.

2.2.3 NIST Cybersecurity Framework

The NIST Cybersecurity Framework organises security activities into five functions: Identify, Protect, Detect, Respond, and Recover [14]. The lab design follows these principles by enabling students to identify vulnerabilities through hands-on exercises and understand corresponding protective measures.

2.3 Prior Academic Research

Academic research consistently validates the effectiveness of hands-on, laboratory-based approaches in cybersecurity education. Conti and Di Pietro [15] demonstrate that challenge-based environments significantly improve skill retention and practical competency compared to purely theoretical instruction. Murphy et al. [16] provide a structured methodology for designing laboratory exercises that are explicitly aligned with curriculum outcomes, validating the need for scenario-specific tasks and integrated assessment. Sponj and Heričko [17] conducted a systematic review of cybersecurity education research, identifying hands-on practice and authentic task design as the two most influential factors in learner engagement and competency development. Al Mohanadi and Furnell [18] further establish a requirements-driven approach to information security assessment laboratories, emphasising that lab tasks must map directly to defined learning outcomes to achieve meaningful pedagogical impact.

These findings directly inform the design choices in this project: each lab is built around scenario-based flags mapped to specific learning objectives, includes an instructor reference sheet, and culminates in an automated assessment mechanism, addressing the gap between platform availability and curriculum integration identified in the prior literature.

2.4 Gap Analysis: What Existing Solutions Miss

Despite the proliferation of cybersecurity training platforms, significant gaps remain:

Gap 1: Curriculum Integration. Most platforms target independent learners or certification preparation rather than integration into specific degree programmes. No major platform explicitly maps pentesting scenarios to the learning outcomes of a particular computer science curriculum.

Gap 2: Institution Control. Effective educational deployment requires institutional control over content, assessment, and infrastructure.

External platforms introduce dependencies and limit customisation, while fully self-hosted solutions often impose high maintenance burdens.

Gap 3: Integrated Network Penetration Testing. Most platforms focus on isolated services or individual techniques. Few provide realistic, multi-system network topologies where reconnaissance informs a complete attack chain and lateral movement is required.

Gap 4: Automation and Scalability. Even classroom-oriented platforms often lack automated deployment, environment reset, and assessment mechanisms, increasing instructor workload and limiting large-scale institutional adoption.

Gap 5: Balance of Guidance and Autonomy. Effective labs must balance sufficient structure to keep students on target with sufficient openness to encourage problem-solving and methodology development.

2.5 The Need for EVE-NG Based Labs

This analysis reveals that while excellent individual components exist—gamified learning (TryHackMe), challenging scenarios (HackTheBox), research-backed content (SEED Labs), specialised domains (GOAD)—no existing solution comprehensively addresses the needs of a university cybersecurity education programme:

- **Curriculum-aligned labs**: Mapped to specific degree learning outcomes
- **Integrated network scenarios**: Multi-system penetration testing with realistic complexity
- **Institutional control**: Locally deployable, customisable, and maintainable
- **Complete automation**: One-command deployment and reset with built-in assessment

- **System-level realism:** Full virtual machine workflows (real OS images, ISO-based installation, native kernel behaviour) that reproduce the technical conditions of professional pentesting practice

The bachelor's thesis project bridges this gap by leveraging EVE-NG [6] to create a comprehensive, reusable, and automatable teaching package for cybersecurity practical labs, synthesising the pedagogical strengths of existing platforms while addressing their limitations through institutional control, curriculum alignment, and automation.

3 Feasibility Study

3.1 Gantt Chart and Task Definition

The project comprises **37 tasks** across 7 categories, totalling 820 hours (20 ECTS × 25 h/ECTS). The detailed phase breakdown and activity descriptions are provided in the Methodology chapter (Chapter 5).

Category	Tasks	Hours
Preparation	4	105h
Analysis	5	49h
Documentation	6	95h
Development	8	178h
Testing	5	123h
Implementation	6	220h
Milestones	3	95h
TOTAL	**37**	**820h**

Table 3.1: Distribution of tasks by category

The complete Gantt chart was built with a custom interactive web tool available at `docs/resources/gantt/index.html` in the project repository.

3.1.1 Critical Path

The project has **13 critical tasks** (277 h total) that directly impact milestone deadlines:

ID	Critical Task	Hours	Deadline
1.4	Pentesting Methodology and Validation	8h	Nov 26, 2025
1.10	Milestone: Preliminary Project	40h	Jan 16, 2026
2.1	Lab 1 Research	14h	Jan 26, 2026
2.2	Lab 2 Research	14h	Jan 31, 2026
2.3	Lab 3 Research – Network Analysis & IR	14h	Feb 7, 2026
2.6	Pre-validation with Pilot Users	20h	Feb 23, 2026
2.6.1	Pre-validation Revision	6h	Mar 1, 2026
3.1	Lab 4 Research	16h	Mar 23, 2026
3.4	Penetration Testing Round 1	30h	Apr 16, 2026
3.5	Penetration Testing Round 2	25h	Apr 30, 2026
3.9	Bachelor's thesis Report Writing I	15h	May 1, 2026
3.10	Bachelor's thesis Report Writing II	40h	May 20, 2026
3.11	Bachelor's thesis Report Writing III	35h	May 25, 2026
TOTAL CRITICAL PATH		**277h**	

Table 3.2: Critical path tasks and deadlines

3.2 Budget

3.2.1 Human Resources

Phase	Hours	Rate	Total
Phase 0: Preparation	105h	15 €/h	1,575 €
Phase 1: Preliminary Project	188h	15 €/h	2,820 €
Phase 2: Interim	212h	15 €/h	3,180 €
Phase 3: Final	315h	15 €/h	4,725 €
TOTAL	**820h**		**12,300 €**

Table 3.3: Human resources cost (junior engineer rate, 15 €/h, theoretical)

3.2.2 Hardware and Software

Resource	Qty	Unit	Total
HomeLab Server (2× Xeon E5-2680v4, 64 GB, HDD+SSD)	1	800€	800€
Development Workstation	1	1,200€	1,200€
Network Equipment	1 set	150€	150€
External Storage	1	100€	100€
SUBTOTAL Hardware			**2,250€**
SUBTOTAL Software			**0€**

Table 3.4: Hardware and software costs (all software is free/open-source)

The server is a one-time investment reused across multiple projects. An additional 32 GB RAM module (~80€) was acquired during the project and is not reflected in the original estimate.

3.2.3 Total Budget

Category	Total
Human Resources	12,300€
Hardware	2,250€
Software	0€
Other (electricity, internet, contingency)	1,020€
TOTAL	**15,570€**

Table 3.5: Complete project budget

3.3 Feasibility Analysis

3.3.1 Technical Feasibility

Phase 0 infrastructure is 100% operational (EVE-NG, Proxmox[19], Parrot OS[20], development environment). All software is open-source and widely adopted in professional cybersecurity training. The required competencies are:

Area	Required Competency	Level
Network Administration	VLANs, routing, firewall rules	Intermediate
Operating Systems	GNU/Linux administration (Debian/Ubuntu)	Intermediate
Virtualisation	EVE-NG topology design, VM management	Advanced
Pentesting	Reconnaissance, exploitation, web vulns	Intermediate
Scripting	Python and Bash for automation	Intermediate
Documentation	LaTeX, technical writing	Intermediate

Table 3.6: Required technical competencies

Risk assessment follows NIST SP 800-30[21]. Table 3.7 summarises the six identified risks with probability, impact, and mitigation strategy.

Risk	Prob.	Impact	Level	Mitigation
EVE-NG Performance Issues	Med	High	HIGH	Dedicated 32 GB RAM; VirtualBox fallback
VM Image Incompatibility	Med	Med	MEDIUM	Early validation; QCOW2 standardisation
Knowledge Gaps (advanced net.)	Med	Med	MEDIUM	Structured learning; tutor consultation
Hardware Degradation	Low	High	MEDIUM	Quarterly diagnostics; S.M.A.R.T. alerts
Data Loss (backup insufficient)	Low	High	MEDIUM	Daily backups; Git + off-site cloud
Configuration Gaps	Med	Med	MEDIUM	Continuous docs; topology exports via Git

Table 3.7: Risk assessment matrix (NIST SP 800-30). No risk is project-blocking.

Conclusion: High technical feasibility. All risks have concrete mitigations and none is project-blocking.

3.3.2 Economic Feasibility

Alternative	Description	Annual Cost
This Project	Custom EVE-NG labs, curriculum-aligned	**0 €/year***
HackTheBox Academy	Commercial labs platform	1,200 €/year
TryHackMe Education	Gamified learning paths	800 €/year
Cybrary for Teams	Video training + labs	1,500 €/year
Custom Enterprise Labs	Outsourced development	8,000–15,000 €

Table 3.8: Cost comparison with commercial alternatives (*excludes initial development)

Conclusion: High economic feasibility. Zero recurring software costs; hardware amortised over 3–5 years; commercially comparable alternatives cost 800–1,500 €/year.

3.3.3 Environmental Feasibility

The project's energy footprint is dominated by the HomeLab server (150 W avg.) and workstation (65 W avg.). Development phase consumption: 188.6 kWh; operational phase (4 sessions × 30 students × 3 h/year): 82.8 kWh/year. A 5 kWp rooftop solar installation at the author's home generated 6,634.9 kWh in 2025 (real data), offsetting approximately 34% of development-phase consumption and reducing CO_2 emissions by 32% relative to grid-only scenarios[22], [23].

Scenario	Description	CO_2/year	With Solar
This Project (ops)	Virtualised shared HomeLab	20.7 kg	14.1 kg
This Project (dev)	Development phase, solar-offset	47.2 kg	32.0 kg
Individual VMs	30 students × personal laptop	180 kg	–
Cloud-Based Labs	AWS/Azure (PUE 1.5)	95 kg	–
Physical Lab	Dedicated hardware switches/routers	450 kg	–

Table 3.9: CO^2 comparison across laboratory approaches

Sustainability measures: VMs suspended when idle; digital-only documentation (3 printed copies required by regulation); modular server

hardware designed for 3–5 year longevity; open-source stack extends usable lifespan of existing hardware.

Conclusion: Good environmental feasibility; significantly lower carbon footprint than physical labs or individual student deployments.

3.3.4 Legal Aspects

Intellectual Property and Licensing

All software (EVE-NG, Parrot OS Security, Metasploitable, DVWA) is licensed under permissive open-source licenses (GPL, MIT, Apache 2.0) that permit educational use, modification, and redistribution. Per UPF regulations (Article 10), intellectual property of the bachelor's thesis belongs to the student; exploitation rights default to the student unless otherwise agreed with the tutor and institution. All citations follow IEEE standards.

Ethical and Legal Considerations for Pentesting

All pentesting activities are conducted within isolated EVE-NG sandboxes with no connection to production networks. Only intentionally vulnerable systems (Metasploitable, DVWA[24]) designed explicitly for training are used as targets. The project is conducted under academic supervision (Pere Vidiella i Catalan) within the scope of the "Introduction to Cybersecurity" curriculum. No personally identifiable information is processed; all scenarios use synthetic data.

Use of Artificial Intelligence Tools

Auxiliary AI tools — both locally deployed language models on the HomeLab and complementary online services — were used strictly to review draft text, detect inconsistencies, and check documentation errors. No AI tool was used to generate original technical content, conclusions, research findings, or design decisions; all creative, analytical, and engineering work is the author's own. This disclosure is

provided in accordance with the academic integrity guidelines applicable to the programme.

Compliance with Academic Regulations

The project follows Tecnocampus ESUPT bachelor's thesis regulations (approved September 30, 2021): original work with IEEE citation standards (Article 11 on plagiarism); structure aligned with Chapter 9 evaluation requirements (continuous assessment: preliminary project proposal, interim, final delivery, tribunal defence); formatting per the ESUPT redaction guide.

3.3.5 Gender and Diversity Perspective

The *Introduction to Cybersecurity* course has no gender or diversity restrictions. In consultation with the responsible professor, the practical barriers to engagement are not demographic but relate to two specific factors:

- **Prior knowledge**: Students with limited background in networking or operating systems may find early labs more demanding. Mitigated through structured guides with progressive hints and a tools reference section in each student PDF.
- **Reading difficulties (dyslexia)**: Dense technical documentation can be harder to process for dyslexic readers. This thesis is typeset in OpenDyslexic; lab documentation uses short paragraphs, descriptive headings, and visually separated code blocks.

All documentation uses gender-neutral language. Fictional accounts and scenario names are culturally neutral and carry no gender or demographic assumptions.

Conclusion: Inclusion measures are grounded in the real barriers identified for this academic context — knowledge scaffolding and typographic accessibility.

Overall Feasibility. The project is feasible across all evaluated dimensions. Technical risks have been mitigated through hardware investment (64 GB RAM, additional module acquired during development). The budget is within academic scope and the cost model is transparent. Legal and ethical obligations are satisfied through open-source licensing and a locally controlled infrastructure. All planned activities fit within the TecnoCampus normative timeline. No identified risk is considered project-blocking.

Full phase task lists (Task IDs, hours, critical flags), the five-phase NIST SP 800-30 risk assessment, and detailed budget breakdowns are provided in Appendix I.

4 Objectives and Scope

Develop, implement, and validate a reusable teaching package of practical cybersecurity labs based on EVE-NG[6], enabling students in the "Introduction to Cybersecurity" course to acquire practical skills in ethical pentesting through realistic and automated scenarios.

4.1 Specific Objectives (Measurable with KPIs)

4.1.1 OE1: Design and Implement EVE-NG Labs

Description: Create four functional and interconnected thematic labs.

Measurable KPIs:

- 4 fully functional .unl topologies (100%)
- Deployment time $\leq$ 2 minutes per lab (95% of cases)
- VM boot success rate $\geq$ 95%
- Compatibility with EVE-NG Community and Professional editions

4.1.2 OE2: Develop Automation Scripts

Description: Develop utility scripts that automate recurring lab management tasks, reducing manual effort for instructors and ensuring repeatable configurations across lab sessions.

Measurable KPIs:

- At least one functional automation script per lab covering a recurring task
- Scripts documented and reusable across different EVE-NG installations

- All scripts tested and verified in the project's EVE-NG environment

4.1.3 OE3: Develop Structured Teaching Material

Description: Develop comprehensive documentation for students and teachers.

Measurable KPIs:

- 4 student lab guides (1 per lab) in PDF format — task description, hints, and tool reference
- 4 instructor resolution guides (1 per lab) in PDF format — step-by-step walkthrough, expected output, and assessment rubric
- Assessment rubric included in each resolution guide with quantifiable criteria
- Lab setup procedure documented for instructors in each resolution guide

4.1.4 OE4: Validate Usability and Educational Effectiveness

Description: Verify that the package meets educational and technical requirements through a reduced pilot group representative of the target student population.

Measurable KPIs:

- Average lab resolution time within the 90–120 minute target window
- All pilot participants able to complete the lab with the provided guide
- User satisfaction score $\geq$ 4/5 (post-session survey)
- Zero blocking technical incidents during pilot sessions

Note: The pilot group comprises 4–5 individuals from the GEISI student population, potentially including participants with documented learning difficulties (e.g. dyslexia), to assess the effectiveness of the accessibility measures incorporated into the lab documentation.

4.2 Definition of the Client and Final User

4.2.1 Principal Client

- **Responsible professor:** Pere Vidiella i Catalan
- **Course:** Introduction to Cybersecurity (GEISI, Tecnocampus)
- **Context:** Undergraduate teaching in practical cybersecurity

4.2.2 Primary End Users

- **GEISI Students:** 25–30 students per edition
- **Profile:** Basic knowledge of networks and operating systems
- **Objective:** Learn ethical pentesting in a practical and safe manner

4.3 Scope Boundaries

Included in this bachelor's thesis:

- Design and implementation of 4 pentesting labs in EVE-NG
- Integration of real vulnerabilities and techniques aligned with OWASP Top 10[12] and CIS Critical Controls[13]
- Utility automation scripts for recurring lab management tasks (e.g. bridge configuration, ISO upload, connectivity checks)
- Complete technical documentation for instructors and future maintainers
- Student guides and learning objectives for each lab

- Assessment rubrics for evaluating student lab completion

Explicitly NOT included (out of scope):

- Integration with third-party Learning Management Systems (LMS) or Moodle
- Development of proprietary lab management software or web interfaces
- Creation of automated grading systems beyond basic script validation
- Commercial licensing or distribution models
- Formal pedagogical evaluation studies (e.g., pre/post learning assessments)
- Creation of labs beyond the four specified domains

4.4 KPIs and Performance Indicators

Category	Metric	Objective	Measurement Method
Technical	Lab deployment time	$\leq$ 2 min	Automated timing
Technical	VM boot success rate	$\geq$ 95%	System logs + validation scripts
Technical	Full reset time	$\leq$ 30 s	Scripts with timestamps
Technical	Scenario reproducibility	100%	Automated tests
Educational	Lab resolution time	90–120 min	Time tracking during pilot
Educational	Lab completion (pilot)	All participants	Observation during pilot sessions
Educational	User satisfaction	$\geq$ 4/5	Post-use survey
Educational	Key concept understanding	$\geq$ 80% correct	Assessment questionnaire
Quality	Documentation coverage	100% of features	Verification checklist
Quality	Critical errors	0	Exhaustive testing
Quality	EVE-NG compatibility	100%	Tests in multiple environments
Quality	Interface usability	$\geq$ 4/5	UX heuristic evaluation

Table 4.1: Key performance indicators for the lab teaching package

Note: KPIs are being measured during the pilot validation phase (April–May 2026) and will be documented in the final report.

Full deliverable specifications per objective, per-category KPI breakdowns, and extended audience analysis are provided in Appendix F.

5 Methodology

This chapter outlines the research strategy, development approach, validation methods, and project lifecycle management employed in this bachelor's thesis. The project integrates principles from software engineering, cybersecurity pedagogy, and empirical research to ensure systematic development and validation of the EVE-NG[6]-based teaching package.

5.1 Research and Development Phases

The project is structured in four distinct phases:

5.1.1 Phase 0: Preparation and Infrastructure (July–November 2025)

Objective: Establish the technical foundation and personal capacity for the project. Activities included HomeLab infrastructure initialisation (Proxmox[19]), development environment setup (VSCode[25], LaTeX[26], Git), and security tools installation (Parrot OS[20], Metasploit[27], Burp Suite[28]).

Duration: 105 hours **Status**: Completed (100%)

5.1.2 Phase 1: Preliminary Project Proposal and Conceptual Design (October 2025–January 2026)

Objective: Define project scope, objectives, and detailed planning. Work covered SMART objectives definition, state-of-the-art analysis of 7 platforms, functional and non-functional requirements specification, pentesting methodology selection (PTES[29], OWASP[30], CEH[3]), risk analysis with 12+ identified risks, resource allocation, and Gantt chart construction with 37 tasks across the critical path.

Duration: 188 hours **Status**: Completed **Milestone**: January 16,

2026

5.1.3 Phase 2: Implementation and Interim Validation (January–March 2026)

Objective: Develop the first three laboratories and conduct preliminary validation. Labs 1–3 were built and tested (reconnaissance, web vulnerabilities, and network/incident response), covering research, topology design, vulnerability implementation, and teaching material production. Informal pilot sessions with 4–5 GEISI students were conducted as each lab was completed, with usability observations, post-session surveys, and feedback-driven adjustments.

Duration: 212 hours **Status:** In progress **Milestone:** March 20, 2026

5.1.4 Phase 3: Completion and Final Validation (March–May 2026)

Objective: Develop Lab 4, complete automation scripts, and conduct final validation. Activities include privilege escalation lab development (CEH framework), two rounds of comprehensive penetration testing across all four labs, utility script development (Bash/Python), final pilot validation with the student group, and full bachelor's thesis report writing and revision.

Duration: 315 hours **Status:** In progress **Final Deadline:** May 27, 2026

5.2 Information Search Strategies

Research draws on academic databases (IEEE Xplore, ACM Digital Library, Scopus, Google Scholar) and primary industry standards: PTES[29], OWASP[30], NIST[14], [31], CIS Controls[13], and EC-Council CEH[3]. Platform and tool documentation was consulted for EVE-NG[32], Parrot OS[20], Metasploit[27], OWASP ZAP[33], and Burp Suite[28]. Primary research uses informal pilot sessions with GEISI students, post-session surveys, and quantitative metrics (time-on-task, completion rates).

5.3 Development Methodology

The project adopts an iterative, agile-inspired approach:

- **Design:** Conceptual architecture and topology planning
- **Implementation:** Lab creation, vulnerability implementation, tooling
- **Validation:** Security testing, educational effectiveness review
- **Iteration:** Feedback incorporation and refinement
- **Documentation:** Technical and pedagogical material preparation

All artefacts are version-controlled in a GitHub repository. Formal documentation is produced in LaTeX (this memory, Volume I) and published as a MkDocs web site for operational reference. Instructor deployment references, startup checklists, and credential tables are compiled separately in the Annexos (Volume II).

5.4 Platform Selection Rationale

A detailed comparative analysis of existing cybersecurity training platforms — including HackTheBox, TryHackMe, Cisco NetAcad, SEED Labs, GOAD, VulnHub, and Hack4u — is presented in the State of the Art chapter. That analysis evaluates each platform across customisation, institutional control, automation, cost, and curriculum alignment, and concludes that EVE-NG is the optimal choice. Its key advantages — open-source licensing, full topology customisation, local deployability, and automation support — directly address the gaps identified in existing platforms.

5.5 Validation and Success Criteria

Success is measured against the KPIs defined in Chapter 4, covering three dimensions: technical performance (deployment time, boot

success rate, reproducibility), educational effectiveness (lab resolution time, completion rate, user satisfaction), and sustainability (documentation coverage, open-source compliance, institutional readiness). Validation is conducted through pilot sessions with a representative group of GEISI students and through automated technical checks at each development milestone.

5.6 Risk Management

Risk	Mitigation Strategy	Owner
Scope creep / Feature additions	Clear scope boundaries and change management	Eloi
EVE-NG compatibility issues	Extensive testing on Community and Pro editions	Eloi
Pilot user dropout	Clear incentives and scheduling flexibility	Pere + Eloi
Performance degradation	Early benchmarking and optimisation iterations	Eloi
Documentation gaps	Peer review and user feedback cycles	Eloi + Faculty
Institutional adoption barriers	Change management plan and training support	Pere

Table 5.1: Key project risks and mitigation strategies

Full phase activity lists, quality assurance procedures, and documentation practices are provided in Appendix G.

6 Functional and Non-Functional Requirements

This chapter specifies the requirements for the EVE-NG-based [6] cybersecurity teaching package: functional requirements (RF) define what the system must do, technological requirements (TR) specify the technical environment, and non-functional requirements (NFR) address quality attributes and compliance.

6.1 Functional Requirements

6.1.1 RF1: Design and Implement EVE-NG Topologies

Description: Create four separate, functional, and interconnected network topologies in EVE-NG format (.unl files) that simulate real-world cybersecurity scenarios.

- **RF1.1:** Four topologies, each targeting a specific pentesting phase:
  - Lab 1: Reconnaissance (passive intelligence gathering)
  - Lab 2: Web application security (active vulnerability discovery)
  - Lab 3: Incident response and network forensics (lateral movement and log analysis)
  - Lab 4: Privilege escalation (post-exploitation scenarios)

Success Criteria: See technical KPIs in Chapter 4 (Table 4.1).

6.1.2 RF2: Implement Vulnerabilities and Attack Scenarios

Description: Implement realistic vulnerabilities aligned with PTES[29], OWASP[30], NIST[14], and CEH[3].

- **RF2.1**: Lab 1 — Reconnaissance: passive tools (Shodan, DNS enumeration), active scanning (Nmap[34], banner grabbing), service discovery and reporting.

- **RF2.2**: Lab 2 — Web Vulnerabilities (OWASP Top 10): SQL injection, command injection, authentication bypass, session flaws, XSS, CSRF, broken access control, and security misconfigurations.

- **RF2.3**: Lab 3 — Incident Response: lateral movement analysis, log forensics (auth.log, bash_history), privilege escalation via sudo misconfiguration, network traffic analysis with Wireshark[35].

- **RF2.4**: Lab 4 — Privilege Escalation (CEH): local privilege escalation, SUID/SGID binaries, sudo misconfigurations, writable cron jobs, capabilities, and post-exploitation persistence.

Success Criteria: Minimum 3–4 exploitable scenarios per lab, all resolvable with documented remediation procedures.

6.1.3 RF3: Create Structured Assessment Materials

Description: Develop assessment rubrics and evaluation materials aligned with the GEISI curriculum [36].

- **RF3.1**: 3–4 specific learning outcomes per lab, mapped to BLOOM's taxonomy (knowledge, application, analysis) and GEISI degree competencies.

- **RF3.2**: Detailed assessment rubric per lab (100 pt scale with explicit criteria), included in the instructor resolution guide.

Success Criteria: 4 student lab guides and 4 instructor guides (resolució PDF), each including a rubric.

6.2 Technological Requirements

6.2.1 TR1: Virtualisation Infrastructure

- **TR1.1**: EVE-NG 5.0+ (Community and Professional editions), deployed as a VM on Proxmox VE; web interface via HTTPS; RESTful API for automation.

- **TR1.3**: Host specifications: 2× Intel Xeon E5-2680v4 (56 logical cores), 64 GB RAM, 2 TB NVMe storage, Gigabit Ethernet.

6.2.2 TR2: Security Tools and Penetration Testing Framework

- **TR2.1**: Attacking platform: Parrot OS Security 6.x [20] (primary). Selected over Kali Linux for lower memory/disk footprint in concurrent EVE-NG deployments, cleaner QEMU integration, and a more approachable environment for students new to security distributions.

- **TR2.2**: Core tools on Parrot OS: Nmap, Metasploit 6.x[27], Burp Suite[28], OWASP ZAP[33], Wireshark[35], tcpdump, Python 3.10+, Bash.

- **TR2.3**: Target images: Ubuntu Server 24.04 LTS, Ubuntu Desktop 24.04 LTS, pfSense CE, VyOS, and the custom SYNAPSE web application (Lab 2).

6.2.3 TR3: Scripting and Automation

- **TR3.1**: Bash and Python 3.10+ for deployment, reset, and validation scripts.

- **TR3.2**: At least one utility script per lab; ISO uploader (`iso_uploader.py`), bridge configuration, and connectivity check scripts; all documented with usage instructions in the repository.

6.3 Non-Functional Requirements

NFR	Attribute	Key Requirements
NFR1	Performance	Deploy $\leq$2 min; VM boot $\leq$90 s (95%); reset $\leq$30 s; VM image $\leq$5 GB; $\leq$6 GB RAM per lab instance
NFR2	Compliance	NIST CSF functions (Identify/Protect/Detect/Respond/Recover); CIS Controls v8; OWASP Top 10 full coverage; PTES + CEH alignment
NFR3	Reliability	MTBF $\geq$100 h; MTTR $\leq$5 min; 100% scenario reproducibility; zero critical bugs at release
NFR4	Security	Full network isolation from production; no outbound internet from labs; RBAC for lab management; mandatory ethics agreement
NFR5	Usability	Faculty deploy time 5–10 min; student prerequisites minimal; WCAG 2.1 Level AA for web interfaces
NFR6	Sustainability	All components open-source or freely available; no vendor lock-in; explicit mapping to GEISI learning outcomes; horizontally scalable

Table 6.1: Non-functional requirements summary

6.4 Requirements Traceability Matrix

ID	Type	Requirement	Scope
RF1.1	Functional	Design 4 EVE-NG topologies	All labs
RF2.1	Functional	Lab 1 reconnaissance scenarios	Lab 1
RF2.2	Functional	Lab 2 OWASP vulnerabilities	Lab 2
RF2.3	Functional	Lab 3 incident response / forensics	Lab 3
RF2.4	Functional	Lab 4 privilege escalation	Lab 4
RF3.1	Functional	Learning outcomes defined	All labs
RF3.2	Functional	Assessment rubrics	All labs
TR1.1	Technical	EVE-NG 5.0+	Platform
TR1.3	Technical	Host 64 GB RAM, 2 TB storage	Infrastructure
TR2.1	Technical	Parrot OS Security as attacking platform	Tools
NFR1	Non-Func.	Deploy $\leq$2 min, reset $\leq$30 s	Performance
NFR2	Non-Func.	NIST / CIS / OWASP / PTES alignment	Compliance
NFR4	Non-Func.	Network isolation + ethics	Security
NFR6	Non-Func.	Open-source, GEISI aligned	Sustainability

Table 6.2: Requirements traceability matrix

Detailed sub-requirements for RF1.2–RF1.4, RF2.5, RF3.3–RF3.4, TR1.2, TR1.4, TR2.4, TR3.3, and the full NFR subsections are provided in

Appendix H.

7 Development

This chapter describes the implementation of the penetration testing lab environment built on EVE-NG. It covers the platform setup, the management of virtual machine images, the lab management interface, the infrastructure problems encountered during development, and an overview of the lab topologies designed for the course. Each individual lab is then described in its own section, with a summary of the network topology, techniques, and problems encountered. Detailed instructor references — including deployment credentials, startup procedures, and flag locations — are compiled in Appendices efapp:lab1– efapp:lab3. The EVE-NG installation procedure is documented in Appendix efapp:eve-setup.

7.1 Platform Setup

The development process began with the installation and validation of EVE-NG [6] as the network emulation platform. Following the official EVE-NG documentation, an initial environment was deployed in order to verify basic functionality: creating simple network topologies, deploying virtual machines with different operating systems, and testing console and remote desktop access before committing to the full lab design.

Several attempts were made to run EVE-NG as a virtual machine on a VMware-based infrastructure available in the initial university environment. Despite VMware being an officially supported hypervisor, recurrent stability and integration issues prevented a reliable setup. After evaluating the available alternatives and in agreement with the project supervisor, the platform was migrated to a dedicated Proxmox VE [19] server. This choice offers better compatibility with the nested virtualisation required by EVE-NG and can be replicated on any institutional server running the same hypervisor.

On top of Proxmox, an EVE-NG virtual machine was deployed following the official installation guidelines, with the networking model

adapted to the Proxmox bridge configuration. Console access via Telnet and graphical access via VNC were validated successfully for all node types used in the labs. Native Wireshark integration was not enabled in this iteration due to conflicts with pre-existing software on the host machine, and was therefore left out of scope for the current version.

7.2 Image Management

7.2.1 Image Selection

Once the platform was stable, a survey of available EVE-NG-compatible images was conducted, prioritising free and open-source operating systems suitable for an academic context. Starting from a broad list of candidate images, the final selection was reduced to a minimal, reusable core for the penetration testing scenarios:

- **VyOS** [37]: used as a router and Layer 3 switch for routing and inter-VLAN scenarios.
- **pfSense CE** [38]: used as the perimeter firewall, providing NAT, DNS resolution, and port forwarding.
- **GNU/Linux distributions** (Ubuntu Server 24.04 [39], Ubuntu Desktop 24.04, Parrot OS [20]): used as target servers, internal workstations, and attacker machines respectively.

This reduced set balances realism and manageability, keeps all software licences aligned with the academic context, and limits the overhead of maintaining multiple image versions across labs.

Parrot OS Security Edition was selected over the more widely known Kali Linux [40] after evaluating both options during image integration. The Kali image proved substantially heavier, required additional QEMU guest configuration to work reliably inside EVE-NG, and produced higher per-instance resource overhead. Parrot OS provides an equivalent toolset — Nmap, Metasploit, Burp Suite, Hydra, Gobuster, and all tools required by the labs — with a lighter footprint and

cleaner EVE-NG integration. An additional consideration was accessibility: Parrot OS includes OpenDyslexic and other accessibility fonts by default, consistent with the inclusive design principles adopted for the lab documentation. This decision is reflected in the requirements specification (Section TR2.1).

7.2.2 Image Organisation in EVE-NG

To ensure consistency and reproducibility, all images were organised under the standard EVE-NG path `/opt/unetlab/addons/qemu/`. Each system was placed in a dedicated directory with a normalised name following the pattern `<type>-<version>` (e.g., `pfsense-2.7.2`, `vyos-1.x-rolling-amd64`, `linux-ubuntu-server-24.04`).

For GNU/Linux systems, the `linux-` prefix was applied consistently (e.g., `linux-ubuntu-desktop-24.04`, `linux-parrot-rolling`), since EVE-NG only makes the *Linux* node category available in the interface when at least one directory with this prefix exists. Each directory contains a bootable image named `cdrom.iso` and a `virtioa.qcow2` disk image sized according to the role of the node.

7.2.3 ISO Upload Automation

The manual workflow for preparing each image — creating the directory, uploading the ISO, renaming it, creating the disk, and fixing permissions — proved repetitive and error-prone during initial experiments. To address this, a Python script named `iso_uploader.py` was developed as part of the project tooling.

The script presents a graphical file picker, uploads the selected ISO files to the EVE-NG server via SCP, and then automates the remaining preparation steps: directory creation under `/opt/unetlab/addons/qemu/`, renaming the ISO to `cdrom.iso`, creation of the `virtioa.qcow2` disk at a size inferred from the image type, and execution of `unl_wrapper -a fixpermissions`. This tool reduces the time required to onboard a new image and enforces a uni-

form directory structure across deployments.

7.3 EVE-NG Lab Management Interface

EVE-NG provides a web-based graphical interface through which administrators and students can manage lab files independently of each other. The *File Manager*, accessible after login, is the main entry point for lab operations and offers several features that are relevant to reproducibility and multi-user deployments:

- **Lab import and export:** Topology files (`.unl`) can be imported by dragging them into the interface. Exported archives preserve the topology, node configurations, and snapshots, making it straightforward to share or back up a complete lab.
- **Folder structure:** Labs can be organised hierarchically (e.g., `/GEISI-Labs/Pentesting/`), which keeps different courses and iterations separate without requiring multiple EVE-NG installations.
- **Multi-lab support:** Several topologies can run simultaneously. The administrator dashboard provides a real-time view of CPU and memory consumption per lab, which is useful for capacity planning on shared infrastructure.

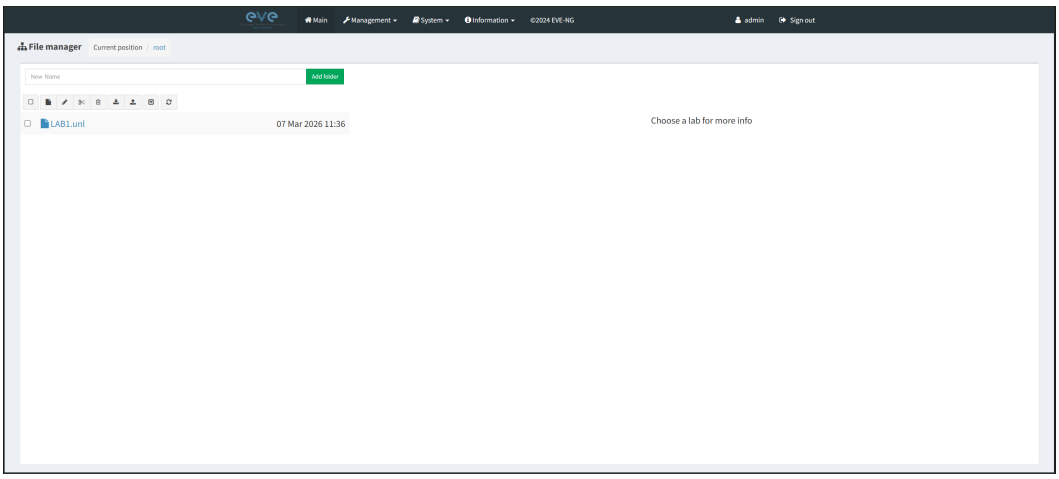

Figure 7.1: EVE-NG File Manager showing the lab directory structure, import controls, and topology thumbnails.

The export format also simplifies distribution: a `.unl` file committed to the project repository can be imported into any compatible EVE-NG instance, reproducing the full topology without manual reconfiguration.

7.4 Infrastructure Problems

Several problems were encountered at the infrastructure level during the setup of the EVE-NG environment. These are distinct from the lab-specific issues documented within each lab section and relate to the underlying platform and tooling.

7.4.1 EVE-NG on VMware

The initial deployment target was a VMware-based server available through the university. Despite EVE-NG being officially listed as compatible with VMware [41] hypervisors, the environment produced recurrent instability: nodes failed to start consistently, VNC sessions dropped without warning, and network connectivity between virtual bridges was unreliable. No stable configuration was found after several iterations. The root cause was not fully identified, but the combination of VMware's nested virtualisation restrictions and the specific hardware and software versions in the university environment appeared to be contributing factors. The decision to migrate to Proxmox resolved all of these issues.

7.4.2 Node Configuration Persistence

A recurring challenge in EVE-NG lab design is ensuring that node configurations survive restarts. By default, runtime network settings applied inside guest virtual machines are lost when the lab is stopped or the EVE-NG host is rebooted. This affected multiple nodes across the lab and required a dedicated solution for each one. The specific mechanisms used are described in the relevant lab sections; the general principle adopted throughout the project was to rely on the native persistence mechanisms of each operating system — such as `systemd-networkd` unit files, NetworkManager connection profiles,

and udev rules — rather than applying settings interactively at the console.

7.4.3 LVM Volume Group Naming Conflict

During a session involving disk operations on guest virtual machine images mounted from the EVE-NG host, a naming conflict arose between the host's LVM volume group and a guest disk's volume group, both named ubuntu-vg. An attempt to resolve the conflict by renaming the host volume group caused the EVE-NG host to fail on the next restart, dropping into an initramfs prompt because the boot configuration still referenced the original group name.

Recovery was performed from the initramfs shell by renaming the volume group back to its original name and reactivating it before resuming the normal boot sequence. As a result of this incident, the rule established for the project is to never rename a volume group belonging to the host operating system. When a naming conflict arises with a guest disk, only the guest volume group is renamed, using a descriptive identifier that includes the lab and node reference.

7.4.4 Memory Capacity Constraint

During concurrent development of the first two laboratories, peak memory consumption on the EVE-NG virtual machine regularly exceeded the initial 32 GB RAM allocation of the host server. Running pfSense, VyOS, Ubuntu Server, Ubuntu Desktop, and Parrot OS simultaneously — alongside other services co-hosted on the same Proxmox infrastructure (monitoring stack, reverse proxy, storage services) — produced memory pressure that caused nodes to be killed by the hypervisor's out-of-memory handler, interrupting development sessions.

An additional 32 GB RAM module was acquired and installed, bringing the host to 64 GB total. This resolved the stability issues and allowed all lab nodes to run concurrently without further intervention. The upgrade was not included in the original project budget estimate; its approximate cost (~80 €) is noted in Chapter 3 as a post-planning

expenditure.

7.5 Lab Topologies

Four penetration testing lab topologies were designed for the course. Each topology is implemented as an EVE-NG `.unl` file and can be imported into any compatible EVE-NG instance. Table 7.1 summarises the four labs, their primary learning objective, and their current implementation status.

Lab	Primary Objective	Status
Lab 1: Reconnaissance & Enumeration	Port scanning, web content discovery, SSH	Implemented
Lab 2: Web Application Vulnerabilities	OWASP Top 10 exploitation	Implemented
Lab 3: Network Analysis	Traffic capture, lateral movement, incident response	Implemented
Lab 4: Privilege Escalation	Local privilege escalation techniques	Pending

Table 7.1: Overview of the four EVE-NG lab topologies designed for the course.

The remainder of this chapter describes each implemented lab in detail, covering the network topology, design decisions, student workflow, and problems encountered during development.

7.6 Lab 1: Reconnaissance and Enumeration

Lab 1 introduces the reconnaissance phase of a penetration test following the PTES framework [29]. Students interact with a fictional target organisation, *PEBCAK Corp*, through a layered topology that includes a perimeter firewall (pfSense CE [38]), an internal router (VyOS [37]), and a target Ubuntu Server 24.04 [39]. Access is exclusively via DNS hostname (`pebcak.local`) resolved through pfSense's Unbound resolver, forcing students to use proper service enumeration before any exploitation attempt. Full topology diagrams, node configuration, student guide, and instructor walk-

through are in Appendix efapp:lab1; interactive documentation at `docs/web/docs/labs/lab1/`.

Network Addressing

Node	Role	Interface	IP	Network
Parrot-OS	Attacker	eth0	10.0.0.10	10.0.0.0/24 (WAN-sim)
pfSense	Firewall/NAT	WAN	10.0.0.1	10.0.0.0/24
pfSense	Firewall/NAT	LAN	192.168.1.1	192.168.1.0/24
VyOS	Router/L3	eth0	192.168.1.2	192.168.1.0/24
VyOS	Router/L3	eth1	10.10.10.1	10.10.10.0/24
Server-PC1	Target	eth0	10.10.10.10	10.10.10.0/24

Table 7.2: Lab 1 IP addressing scheme

Techniques and Framework Alignment

Technique	Description	Reference
DNS resolution	Hostname-to-IP via Unbound resolver (RFC 1035 [42])	PTES Recon
Port scanning	TCP SYN scan with Nmap (RFC 793 [43])	PTES Recon
Service enumeration	Banner grabbing, version detection	PTES Recon
Web content discovery	Directory brute-force with Gobuster	OWASP WSTG
Credential retrieval	Exposed credentials in web content	OWASP A05
SSH access	Password-based login (RFC 4252 [44])	PTES Exploit

Table 7.3: Lab 1 techniques and framework mapping

7.6.1 Problems Encountered and Solutions

VNC Keyboard Input on QEMU Nodes

EVE-NG provides VNC console access to QEMU nodes, but certain key combinations — most notably `Ctrl+C` and special characters such as |, @, and { — are not reliably forwarded by the browser-based VNC client or by the `vncdo` command-line tool. The root cause is

that vncdo sends key symbols rather than raw key codes, and the VNC server's key mapping does not cover all combinations used in a penetration testing context.

The solution adopted was to inject keystrokes directly via the QEMU monitor socket, using Python's raw socket interface to send `sendkey` commands. This bypasses the VNC protocol entirely and ensures that arbitrary key sequences, including those involving modifier keys, are delivered correctly to the guest.

pfSense DNS Resolver WAN Interface

After initial configuration, DNS queries from the attacker machine were not being resolved by pfSense's Unbound resolver. The issue was that Unbound, by default, only listens on the LAN interface, so queries arriving on the WAN-side interface were silently discarded. The fix was to explicitly add the WAN interface to Unbound's listening configuration under *Services* → *DNS Resolver* → *Network Interfaces*.

Unbound Startup Configuration Path

On subsequent restarts, the Unbound configuration that had been applied via the GUI was not being preserved, causing DNS resolution to fail after every reboot. Investigation revealed that pfSense's startup scripts write the Unbound configuration to `/var/unbound/unbound.conf` at boot time, overwriting any manual edits. The correct approach is to configure all resolver settings exclusively through the GUI or through pfSense's `/etc/inc/` PHP hooks, never by editing the generated configuration file directly.

VyOS Source NAT Requirement

Traffic routed through VyOS from the attacker to the target was being dropped because the target's default gateway was pfSense, not VyOS. Return packets were therefore routed back via pfSense rather than through VyOS, breaking the connection. The fix was to

configure Source NAT (masquerade) on VyOS for traffic entering the internal segment, ensuring that return traffic would follow the same path back through the VyOS router.

Results

Flag	Location	Technique
FLAG{recon_enum_success}	`/flag.txt` on Server-PC1 via SSH	Credential reuse
FLAG{pebcak_firewall_pivot}	pfSense admin panel (optional)	NAT pivot

Table 7.4: Lab 1 flags and acquisition techniques

7.7 Lab 2: Web Application Vulnerabilities — SYNAPSE Intelligence Portal

Lab 2 places students in front of a custom two-tier web application, *SYNAPSE Intelligence Portal*, deployed on a segmented DMZ architecture. The scenario is aligned with the OWASP Top 10 [12] and chains multiple vulnerabilities across two servers: Server-A hosts the portal front-end with authentication and reporting features; Server-B provides an internal monitoring API accessible only via Server-A. Students must complete a progressive exploitation chain — stored XSS, cookie theft, SQL injection, and command injection — to collect all three flags. Full topology, source code, credentials, and startup procedure are in Appendix efapp:lab2; interactive documentation at `docs/web/docs/labs/lab2/`.

Network Addressing

Node	Role	Interface	IP	Network
Parrot-OS	Attacker	eth0	10.0.2.10	10.0.2.0/24 (WAN-sim)
pfSense-LAB2	Firewall/NAT/DNAT	WAN	10.0.2.1	10.0.2.0/24
pfSense-LAB2	Firewall/NAT	LAN	192.168.2.1	192.168.2.0/24
VyOS-LAB2	DMZ Router	eth0	192.168.2.2	192.168.2.0/24
VyOS-LAB2	DMZ Router	eth1	10.0.3.1	10.0.3.0/24 (DMZ)
Server-A	Web front-end	eth0	10.0.3.10	10.0.3.0/24
Server-B	Internal API	eth0	10.0.3.20	10.0.3.0/24

Table 7.5: Lab 2 IP addressing scheme

OWASP Coverage and Vulnerability Chain

Step	OWASP	Vulnerability	Flag
1	A03:2021	Stored XSS in report field $ightarrow$ session cookie theft	–
2	A07:2021	Cookie injection $ightarrow$ admin panel access	–
3	A03:2021	SQL injection in search $ightarrow$ credential dump	FLAG{synapse_sqli_creds_dumped}
4	A01:2021	Lateral movement to Server-B via stolen creds	FLAG{synapse_pivot_server_b}
5	A03:2021	Command injection in monitoring API	FLAG{synapse_rce_achieved}

Table 7.6: Lab 2 OWASP Top 10 vulnerability chain

7.7.1 Problems Encountered and Solutions

pfSense Block Private Networks and Reply-To

Two pfSense default behaviours blocked traffic in the lab topology. First, the *Block private networks* option on the WAN interface was enabled by default, which caused pfSense to silently discard packets arriving from the simulated attacker network (RFC 1918 space [45]).

Second, the *Reply-to* feature added return routes in `pf` rules that conflicted with VyOS's routing table, causing asymmetric routing. Both were disabled: the private network block via the WAN interface settings, and reply-to via a per-rule option in the firewall rule editor.

VNC Keyboard Input on pfSense

The same VNC key injection issue documented in Lab 1 reappeared on pfSense console sessions, particularly when entering characters required by the pfSense shell (`|`, `&`, `$`). The QEMU monitor raw socket approach was applied identically.

Server-A Volume Group Conflict

When mounting the Server-A QCOW2 image from the EVE-NG host to pre-stage application files, the host's own LVM volume group (`ubuntu-vg`) conflicted with the guest disk's identically named group. Activating both simultaneously caused the host's logical volumes to be shadowed. The solution was to rename the guest volume group to `synapse-vg` using `vgrename` before activating it on the host, then rename it back after dismounting.

Server-B Installer Keyboard Layout

During the Server-B installation, the Ubuntu installer defaulted to a Spanish keyboard layout due to the locale of the EVE-NG host, causing mismatched characters when entering the initial credentials. Corrected by explicitly selecting the `en_US` keyboard layout at the installer prompt and verified with a `/dev/urandom`-seeded password test before proceeding.

Static Route Gateway Reference

VyOS rejected a static route configuration that referenced a gateway IP not directly reachable on any configured interface. The issue was that the intended next-hop was on a subnet not yet assigned to any VyOS interface at the time the route was applied. Resolved by defining the interface address before the static route, and by using `commit` and `save` in the correct order within the VyOS configuration session.

7.8 Lab 3: Incident Response and Log Forensics — HELIX Systems

Lab 3 inverts the student's role: instead of attacking, they act as a security analyst responding to an active incident at *HELIX Systems*. The lab environment comes pre-staged with evidence of a simulated intrusion — brute-force SSH attempts, a successful login, privilege escalation via `sudo` misconfiguration, backdoor account creation, and lateral movement to a database server. Students must reconstruct the attack timeline from system logs (`auth.log`, `bash_history`) and identify the attacker's actions and impact, following a methodology aligned with NIST SP 800-61 [46]. Full evidence walkthrough, account table, and attack timeline are in Appendix efapp:lab3; interactive documentation at `docs/web/docs/labs/lab3/`.

Network Addressing

Node	Role	Interface	IP	Network
Parrot-OS	Analyst station	eth0	10.0.4.10	10.0.4.0/24 (WAN-sim)
pfSense-LAB3	Firewall/DNAT	WAN	10.0.4.1	10.0.4.0/24
pfSense-LAB3	Firewall	LAN	192.168.3.1	192.168.3.0/24
VyOS-LAB3	Internal router	eth0	192.168.3.2	192.168.3.0/24
VyOS-LAB3	Internal router	eth1	10.0.5.1	10.0.5.0/24 (corp)
Server-Web	Compromised web	eth0	10.0.5.10	10.0.5.0/24
Server-DB	Internal DB	eth0	10.0.5.20	10.0.5.0/24

Table 7.7: Lab 3 IP addressing scheme

MITRE ATT&CK Coverage

MITRE ID	Technique	Evidence in lab
T1110.001	Brute Force: Password Guessing	`auth.log` failed attempts
T1078	Valid Accounts	`devops` account compromise
T1548.003	Sudo and Sudo Caching	`find` NOPASSWD in sudoers
T1136.001	Create Account: Local	Backdoor `sysbackup` account
T1021.004	Remote Services: SSH	Lateral movement to Server-DB
T1005	Data from Local System	`/opt/helix/data/` exfil marker

Table 7.8: Lab 3 MITRE ATT&CK technique coverage

7.8.1 Problems Encountered and Solutions

LVM Volume Group Naming Conflict

When pre-staging Server-Web's evidence files from the EVE-NG host, the same volume group conflict from Lab 2 recurred: both the host and the guest disk used `ubuntu-vg`. The resolution procedure was identical — renaming the guest VG to `helix-web-vg` before activation — and the naming rule was formally added to the project's infrastructure runbook to prevent recurrence across future labs.

Read-only nbd Device

On one occasion, the QCOW2 image mounted via `qemu-nbd` was presented as read-only despite the `-readwrite` flag. Investigation revealed that a previous mount session had not been cleanly disconnected (`qemu-nbd --disconnect`), leaving the kernel's NBD client in an inconsistent state. The fix was to explicitly disconnect any stale NBD device before re-mounting, and to add a pre-mount check to the project's image manipulation scripts.

analyst Group Membership for Log Access

The student account (`analyst`) was created with restricted permissions so that it could not read system logs directly as root. However, `/var/log/auth.log` requires membership of the `adm` group on Ubuntu. The pre-staging script was updated to add `analyst` to `adm` at image preparation time, keeping the educational constraint (no `sudo` to `root`) while allowing access to the evidence files that are the focus of the lab.

pfSense Console Access via QEMU Monitor

The pfSense serial console was inaccessible through the standard EVE-NG Telnet interface due to a configuration mismatch between the QEMU serial port definition and pfSense's expected console device (`ttyu0` vs `ttyS0`). Console access was recovered by connecting directly to the QEMU monitor socket and issuing a `sendkey` sequence to navigate the pfSense boot menu, where the console port assignment could be corrected persistently.

Bibliography

[1] "HackTheBox - cybersecurity training platform." Online pentesting platform with CTF challenges. Accessed: 4 March 2026, HackTheBox Ltd. (2024), [Online]. Available: https://www.hackthebox.com/.

[2] "TryHackMe - learn cybersecurity through hacking." Gamified cybersecurity training platform. Accessed: 4 March 2026, TryHackMe. (2024), [Online]. Available: https://tryhackme.com/.

[3] "CEH certified ethical hacker exam objectives." Certification framework. Accessed: 4 March 2026, EC-Council. (2024), [Online]. Available: https://www.eccouncil.org/programs/certified-ethical-hacker-ceh/.

[4] "OSCP certification guide." Professional pentesting certification. Accessed: 4 March 2026, Offensive Security. (2024), [Online]. Available: https://www.offensive-security.com/pwk-oscp/.

[5] Centre Universitari TecnoCampus. "Centre universitari tecnocampus — official website." Accessed: 2025, Centre Universitari TecnoCampus. (2024), [Online]. Available: https://www.tecnocampus.cat.

[6] "EVE-NG - emulated virtual environment next generation." Network simulation platform - primary platform for TFG. Accessed: 4 March 2026, EVE-NG Community. (2025), [Online]. Available: https://www.eve-ng.net/.

[7] "SEED labs - hands-on lab exercises." Academic cybersecurity lab exercises. Accessed: 4 March 2026, SEED Project. (2024), [Online]. Available: https://seedsecuritylabs.org/.

[8] "GOAD - game of active directory." Vulnerable Active Directory lab environment. Accessed: 4 March 2026, GOAD Project. (2024), [Online]. Available: https://github.com/Orange-Cyberdefense/GOAD.

[9] "VulnHub - vulnerable machine community." Community-driven vulnerable virtual machines. Accessed: 4 March 2026, VulnHub Project. (2024), [Online]. Available: https://www.vulnhub.com/.

[10] "Hack4U - cybersecurity training platform." Spanish cybersecurity training platform with practical labs. Accessed: 4 March 2026, Hack4U. (2026), [Online]. Available: https://hack4u.io/.

[11] "Cisco networking academy." Official networking curriculum. Accessed: 4 March 2026, Cisco Systems Inc. (2024), [Online]. Available: https://www.netacad.com/.

[12] "OWASP top 10 2021: A03:2021 – injection." Accessed: 5 January 2026, Open Web Application Security Project. (2021), [Online]. Available: https://owasp.org/Top10/.

[13] "CIS critical controls v8.0." Framework for defensive security. Accessed: 8 April 2026, Center for Internet Security. (2023), [Online]. Available: https://www.cisecurity.org/cis-controls/.

[14] "Framework for improving critical infrastructure cybersecurity." Version 1.1. Accessed: 8 April 2026, National Institute of Standards and Technology. (2022), [Online]. Available: https://www.nist.gov/cyberframework/framework.

[15] G. Conti and R. Di Pietro, "Hands-on cybersecurity: A challenge-based approach," *IEEE Security & Privacy*, vol. 17, no. 2, pp. 19–27, 2019. DOI: 10.1109/MSEC.2019.2891308.

[16] C. Murphy, R. Fitzpatrick, and D. Mac Cormac, "Design of laboratory exercises for cybersecurity curriculum," in *Proceedings of the 2018 IEEE Conference on Cyber Security and Data Protection (CSDP)*, 2018, pp. 1–8.

[17] M. Sponj and M. Heričko, "Cybersecurity education: A systematic review," *Journal of Educational Computing Research*, vol. 59, no. 4, pp. 528–562, 2021. DOI: 10.1177/0735633120968370.

[18] H. Al Mohanadi and S. Furnell, "A requirements-driven approach to designing an assessment laboratory for information security," *Journal of Educational Computing Research*, vol. 54, no. 4, pp. 469–494, 2016. DOI: 10.1177/0735633115623065.

[19] "Proxmox virtual environment." Open-source hypervisor and virtualization platform. Accessed: 4 March 2026, Proxmox Server Solutions. (2024), [Online]. Available: https://www.proxmox.com/.

[20] "Parrot os - security distribution." Debian-based Linux distribution for security testing. Accessed: 8 April 2026, Parrot Security. (2024), [Online]. Available: https://www.parrotsec.org/.

[21] "Guide for conducting risk assessments." Accessed: 4 March 2026, National Institute of Standards and Technology. (2012), [Online]. Available: https://nvlpubs.nist.gov/nistpubs/Legacy/SP/nistspecialpublication800-30r1.pdf.

[22] "Spanish electricity grid carbon intensity data." Official electricity generation and emissions data. Accessed: 4 March 2026, Red Eléctrica de España. (2024), [Online]. Available: https://www.ree.es/.

[23] S. Murugesan, "Harnessing green IT: Principles and practices," *IT Professional*, vol. 10, no. 1, pp. 24–33, 2008.

[24] "DVWA - damn vulnerable web application." Intentionally vulnerable web application for training. Accessed: 4 March 2026, DVWA Project. (2026), [Online]. Available: https://github.com/digininja/DVWA.

[25] "Visual Studio Code - code editor." Primary code editor for development. Accessed: 4 March 2026, Microsoft Corporation. (2026), [Online]. Available: https://code.visualstudio.com/.

[26] "LaTeX - document preparation system." Typesetting system for technical documents. Accessed: 4 March 2026, LaTeX Project. (2026), [Online]. Available: https://www.latex-project.org/.

[27] "Metasploit framework documentation." Exploitation framework. Accessed: 4 March 2026, Metasploit Project. (2025), [Online]. Available: https://docs.metasploit.com/.

[28] "Burp suite community edition." Web application security testing tool. Accessed: 4 March 2026, PortSwigger Web Security. (2024), [Online]. Available: https://portswigger.net/burp/communitydownload.

[29] "Penetration testing execution standard (ptes)." Industry-standard pentesting framework. Accessed: 4 March 2026, PTES Executive Summary. (2014), [Online]. Available: http://www.pentest-standard.org/.

[30] "OWASP web security testing guide (wstg)." Version 4.2 - Comprehensive testing methodology. Accessed: 4 March 2026, Open Web Application Security Project. (2023), [Online]. Available: https://owasp.org/www-project-web-security-testing-guide/.

[31] "Technical security testing." Guidelines for security testing. Accessed: 8 April 2026, National Institute of Standards and Technology. (2008), [Online]. Available: https://nvlpubs.nist.gov/nistpubs/Legacy/SP/nistspecialpublication800-115.pdf.

[32] "EVE-NG official documentation." Complete platform documentation. Accessed: 4 March 2026, EVE-NG Community. (2025), [Online]. Available: https://www.eve-ng.net/index.php/documentation.

[33] "OWASP ZAP - Web Application Security Scanner." Automated security scanning and testing. Accessed: 4 March 2026, Open Web Application Security Project. (2024), [Online]. Available: https://www.zaproxy.org/.

[34] "Nmap official documentation." Network discovery and security auditing tool. Accessed: 4 March 2026, Nmap Project. (2024), [Online]. Available: https://nmap.org/.

[35] "Wireshark - network protocol analyzer." Packet capture and network analysis tool. Accessed: 4 March 2026, Wireshark Foundation. (2024), [Online]. Available: https://www.wireshark.org/.

[36] "Grau en enginyeria informàtica de gestió i sistemes d'informació (GEISI)." Bachelor program curriculum and learning outcomes. Accessed: 4 March 2026, TecnoCampus - Universitat de Barcelona. (2024), [Online]. Available: https://www.tecnocampus.cat/grau/grau-en-enginyeria-informatica-de-gestio-i-sistemes-dinformacio.

[37] "VyOS - open source router and firewall." Open-source network OS for routing and firewalling. Accessed: 8 April 2026, VyOS Project. (2024), [Online]. Available: https://vyos.io/.

[38] "pfSense community edition." Open-source firewall and router platform based on FreeBSD. Accessed: 8 April 2026, Netgate. (2024), [Online]. Available: https://www.pfsense.org/.

[39] "Ubuntu server and desktop." Open-source Linux distribution. Version 24.04 LTS used for server and workstation nodes. Accessed: 8 April 2026, Canonical Ltd. (2024), [Online]. Available: https://ubuntu.com/.

[40] "Kali linux official documentation." Primary penetration testing distribution. Accessed: 4 March 2026, Kali Linux. (2025), [Online]. Available: https://www.kali.org/docs/.

[41] "VMware vsphere hypervisor (ESXi)." Enterprise hypervisor. Accessed: 4 March 2026, VMware Inc. (2024), [Online]. Available: https://www.vmware.com/products/esxi-and-esx.html.

[42] P. Mockapetris, *RFC 1035: Domain Names — Implementation and Specification*, IETF, 1987. [Online]. Available: https://www.rfc-editor.org/rfc/rfc1035.

[43] J. Postel, *RFC 793: Transmission Control Protocol*, IETF, 1981. [Online]. Available: https://www.rfc-editor.org/rfc/rfc793.

[44] T. Ylonen and C. Lonvick, *RFC 4252: The Secure Shell (SSH) Authentication Protocol*, IETF, 2006. [Online]. Available: https://www.rfc-editor.org/rfc/rfc4252.

[45] Y. Rekhter, B. Moskowitz, D. Karrenberg, G. J. de Groot, and E. Lear, *RFC 1918: Address Allocation for Private Internets*, IETF, 1996. [Online]. Available: https://www.rfc-editor.org/rfc/rfc1918.

[46] "Computer security incident handling guide." Primary NIST guide for incident response. Accessed: 27 April 2026, National Institute of Standards and Technology. (2012), [Online]. Available: https://nvlpubs.nist.gov/nistpubs/SpecialPublications/NIST.SP.800-61r2.pdf.

A EVE-NG Installation on Proxmox VE

This appendix documents the configuration used to deploy EVE-NG Community Edition on the Proxmox VE server used for this project. The procedure has been validated and the same steps can be used to reproduce the environment on any compatible Proxmox host. Screenshots and extended notes are available in the companion web documentation at `docs/web/docs/guides/eve_ng_install_proxmox/`.

Virtual Machine Requirements

Parameter	Value used in this project
Hypervisor	Proxmox VE 8.x
CPU type	`host` (required for nested virtualisation)
vCPU cores	16 (minimum 8)
RAM	32 GB per EVE-NG instance (host upgraded to 64 GB total)
Disk	100 GB VirtIO, SSD-backed Proxmox pool
SCSI controller	VirtIO SCSI
Network	VirtIO, bridged on `vmbr0` (management); `pnet0`–`pnetN` as cloud bridges
BIOS	SeaBIOS (default; OVMF also works)

Table A.1: EVE-NG VM configuration parameters

Installation Steps

1. Download the EVE-NG Community Edition ISO from the official website.
2. In Proxmox, create a new VM with the parameters in Table A.1. Ensure the CPU type is set to `host` so that virtualisation extensions (VT-x / AMD-V) are exposed to the guest.
3. Boot from the ISO and follow the Ubuntu Server installation wizard. Assign a static IP address and install the OpenSSH server during setup.

4. After the first reboot, follow the EVE-NG Community post-install wizard (`/opt/unetlab/wrappers/unl_wrapper -a postinstall`) and configure the management interface.
5. Run `apt update && apt upgrade -y` and reboot once more to ensure all EVE-NG patches are applied.
6. Access the web interface at `https://<EVE-NG-IP>/` and change the default credentials immediately.

Proxmox Network Configuration

Two bridge types are used:

- `vmbr0` **(bridged)** — management access from the LAN; also used as `pnet0` inside EVE-NG for labs that require connectivity to the homelab network (e.g., Lab 1).
- `vmbr1` **(NAT)** — optional internet-facing bridge to keep lab traffic isolated from the LAN while allowing outbound connectivity.

Post-Install Checklist

- ☐ CPU type set to `host` in Proxmox VM options.
- ☐ VM autostart enabled in Proxmox.
- ☐ Snapshot taken after successful EVE-NG installation.
- ☐ Default admin password changed.
- ☐ OS Client Side installed on instructor workstation (HTML5 console option does not require it).
- ☐ Image directories created under `/opt/unetlab/addons/qemu/` and permissions fixed with `unl_wrapper -a fixpermissions`.

B Lab 1: PEBCAK Corp — Instructor Reference

This appendix provides the deployment reference for Lab 1 (Reconnaissance and Enumeration). It is intended for instructors and system administrators responsible for setting up and resetting the lab environment. The student-facing guide and interactive walkthrough are available at `docs/web/docs/labs/lab1/`.

Credentials and Access

Node	Username	Password	Access method
pfSense	admin	pfsense	Web UI / SSH
VyOS	vyos	vyos	EVE-NG console
Server (PEBCAK)	blackmesa	`!Bl4kM3s$`	SSH (via pfSense DNAT)
PC1	ubuntu	ubuntu	EVE-NG console / VNC
Parrot OS	parrot	parrot	EVE-NG console / VNC

Table B.1: Lab 1 credentials (instructor reference — do not distribute to students)

Network Segments

Segment	Subnet	Gateway	Purpose
Homelab / WAN	192.168.0.0/24	192.168.0.1	Parrot attacker + pfSense WAN
Net-Link	172.16.0.0/30	pfSense vtnet0	pfSense ↔ VyOS uplink
Users LAN	192.168.10.0/24	192.168.10.x	PC1 internal workstation
Servers LAN	192.168.20.0/24	192.168.20.1	PEBCAK Corp server segment

Table B.2: Lab 1 network segments

Lab Startup Checklist

1. Start all nodes from the EVE-NG web interface in order: pfSense → VyOS → Server → PC1 → Parrot.

2. Run the bridge script on the EVE-NG host (lost on reboot; persistent via `udev` rule):

```
ip addr add 192.168.20.x/24 dev vnet0_2
ip addr add 192.168.10.x/24 dev vnet0_3
ip route add 172.16.0.0/30 via 192.168.20.1
```

3. Verify DNS resolution from Parrot: the DNS server on Parrot must point to pfSense's WAN IP so that `lab1` resolves.

```
# On Parrot:
curl http://lab1          # Should return PEBCAK Corp HTML
```

4. Confirm flag is in place on the Server node:

```
sshpass -p '!Bl4kM3s$' ssh blackmesa@192.168.20.x cat ~/flag.txt
# Expected: FLAG{p3bc4k_s3rv3r_0wn3d}
```

Expected Student Workflow

Step	Action	Tool
1	Navigate to `http://lab1` in Parrot browser	Firefox
2	Inspect page source; find HTML comment: `<!-- sometimes simplify and search -->`	Browser devtools
3	Access `http://lab1/pebcak.html`; read SSH credentials	Browser
4	`ssh blackmesa@lab1` (pfSense DNAT forwards TCP 22)	SSH
5	`cat ~/flag.txt` — retrieves Flag 1 and pfSense credentials	Terminal
6	SSH to pfSense from Server: `ssh admin@172.16.x.x`	SSH (optional)

Table B.3: Lab 1 expected student workflow

Flags

Flag	Value	Location
Flag 1	`FLAG{p3bc4k_s0me_b1ack3d}`	`/home/blackmesa/flag.txt` on Server
Flag 2	`FLAG{pebcak_fs_revealing_panel}`	Reveal admin panel (optional bonus)

Table B.4: Lab 1 flags

C Lab 2: SYNAPSE Intelligence Portal — Instructor Reference

This appendix provides the deployment reference for Lab 2 (Web Application Vulnerabilities). It is intended for instructors and administrators. The application source code, Docker Compose files, and interactive walkthrough are at `docs/web/docs/labs/lab2/`.

Credentials and Access

System	Username	Password	Role
pfSense-LAB2	admin	pfsense	Firewall management
VyOS-LAB2	vyos	vyos	Router console
Server-A OS	ubuntu	S3rv3rA	SSH access
Server-A Portal	admin	Admin@Synapse2024	Web application admin
Server-A Portal	guest	guest123	Web application guest
Server-B OS	ubuntu	S3rv3rB	SSH access
Server-B DataVault	operator	D4t4V4ult#2024	API access
Parrot-LAB2	lab2	L4b2	Attacker node

Table C.1: Lab 2 credentials (instructor reference — do not distribute to students)

Vulnerability Chain and Flag Locations

Step	OWASP	Vuln	Implementation	Flag
1	A03:2021	Stored XSS	`|safe` filter in Jinja2 template; `/comments` endpoint	—
2	A07:2021	Cookie theft	`HttpOnly=False`; victim bot (Playwright) auto-visits comments	—
3	A07:2021	Broken auth	Cookie format `ID:ROLE:USERNAME`; unsigned; forgeable	—
4	A03:2021	SQL injection	`/search?q=` uses string concatenation	FLAG 1
5	A01:2021	Admin access	`/admin` panel reveals Server-B credentials	FLAG 2
6	A08:2021	YAML deser.	Server-B `/preview` uses `yaml.UnsafeLoader`	—
7	A08:2021	RCE	`!!python/object/apply:os.system` payload via POST	FLAG 3

Table C.2: Lab 2 vulnerability chain and flag locations

Lab Startup Procedure

1. After EVE-NG host reboot, restore bridge addresses:

```
bash /usr/local/bin/lab2-bridges-up.sh
```

2. Start the SYNAPSE Portal on Server-A:

```
sshpass -p S3rv3rA ssh ubuntu@192.168.30.x
cd /opt/synapse && docker-compose up -d
docker-compose ps   # expect: flask, nginx, victim — all Up
```

3. Start the DataVault on Server-B:

```
sshpass -p S3rv3rB ssh ubuntu@192.168.30.x   # Server-B IP
cd /opt/datavault && sudo docker compose up -d
sudo docker compose ps   # expect: datavault-flask Up on :80
```

4. Configure Parrot static IP and verify connectivity:

```
# On Parrot (10.0.40.x/24, GW 10.0.40.x):
nmcli con mod "Wired connection 1" ipv4.method manual \
  ipv4.addresses 10.0.40.x/24 ipv4.gateway 10.0.40.x
```

```
nmcli con up "Wired connection 1"
curl http://192.168.30.x   # Should return SYNAPSE Portal HTML
```

Note: pfSense-LAB2 is installed in UEFI mode. If it fails to boot, use the start script: `/usr/local/bin/pfsense-lab2-start.sh`. Server-A uses `docker-compose` (v1, hyphen); Server-B uses `docker compose` (v2, space).

D Lab 3: HELIX Systems — Instructor Reference

This appendix provides the deployment and pre-staging reference for Lab 3 (Incident Response and Log Forensics). It documents all accounts, the pre-staged attack timeline, and the evidence artefacts that students are expected to find. The step-by-step investigation walkthrough is at `docs/web/docs/Labs/Lab3/walkthrough/`.

Accounts

System	Username	Password	Role
pfSense-LAB3	admin	pfsense	Firewall management
VyOS-LAB3	vyos	vyos	Router console
Server-Web	analyst	Analyst@2024	**Student entry account**
Server-Web	devops	D3vOps#2023	Compromised account (brute-forced)
Server-Web	maint	—	Backdoor account (attacker-created)
Server-DB	analyst	Analyst@2024	Lateral access
Server-DB	devops	D3vOps#2023	Compromised (password reuse)

Table D.1: Lab 3 accounts (instructor reference)

Pre-Staged Attack Timeline

Time	Host	Event
03:05–03:16	Server-Web	25 failed SSH attempts for `devops` from `185.220.101.47`
03:17	Server-Web	Successful SSH login as `devops`
03:18	Server-Web	`sudo /usr/bin/find` — privilege escalation to root via GTFOBins
03:19	Server-Web	`useradd maint` + `/etc/sudoers.d/maint` — backdoor + persistence
03:21	Server-DB	SSH login as `devops` from `192.168.50.10` (lateral movement)
03:22	Server-DB	Access to `clients.db`; `.exfil_marker` created

Table D.2: Lab 3 pre-staged attack timeline

Evidence Artefact Locations

Artefact	Path	Host
SSH brute-force log	/var/log/auth.log	Server-Web
Privilege escalation	/var/log/auth.log (sudo entry)	Server-Web
Backdoor account	/etc/passwd, /etc/sudoers.d/maint	Server-Web
Sudoers misconfiguration	/etc/sudoers.d/devops	Server-Web
Bash history	/home/devops/.bash_history	Server-Web
Lateral movement log	/var/log/auth.log	Server-DB
Exfiltration marker	/opt/helix/data/.exfil_marker	Server-DB
Sensitive data	/opt/helix/data/clients.db	Server-DB

Table D.3: Lab 3 evidence artefact locations

Student Entry Point

Students connect via SSH to pfSense's WAN IP; pfSense DNAT forwards TCP 22 directly to Server-Web:

```
ssh analyst@<pfSense-WAN-IP>
# Password: An@lyst2024
# Lands on: analyst@helix-web:~$
```

No offensive tools are required. Students use only standard Unix utilities: `grep`, `awk`, `less`, `cat`, `find`, `journalctl`.

Lab Reset Procedure

To restore the lab to its initial forensic state after a student session:

```
# Restore EVE-NG snapshot to pre-exercise state
# (from EVE-NG host or web interface)
unl_wrapper -a ...    # see web docs for exact snapshot name
```

Alternatively, revert both Server-Web and Server-DB QCOW2 images from the pre-staged backup stored at `/opt/unetlab/addons/qemu/lab3-backups/`.

E Cybersecurity Training Platform Descriptions

This appendix provides detailed descriptions of the seven cybersecurity training platforms reviewed in Chapter 2. Each subsection discusses the platform's approach, strengths, and limitations relevant to curriculum-aligned institutional deployment. A comparative summary is provided in Table 2.1.

E.1 HackTheBox

HackTheBox [1] is one of the most widely recognised online platforms for penetration testing practice, offering a large library of intentionally vulnerable machines and Capture-the-Flag challenges categorised by difficulty. The platform benefits from a strong community and industry recognition, making it a common supplementary resource for professional certifications. However, its challenges are isolated: there is no network topology connecting systems, no formal curriculum mapping to specific degree programmes, and the platform cannot be deployed on institutional infrastructure, which limits customisation and assessment integration.

E.2 TryHackMe

TryHackMe [2] adopts a gamified, beginner-friendly approach, combining structured learning paths with guided interactive labs accessible entirely from a browser. Its low barrier to entry and progressive structure make it effective for initial skill development. However, the platform is externally managed and subscription-dependent, scenarios are simplified to ensure completion within short timeframes, and it does not support local institutional deployment or integration with academic learning management systems.

E.3 Cisco Networking Academy

Cisco Networking Academy [11] provides institution-backed curriculum aligned with professional certifications (CCNA, CyberOps) and uses Packet Tracer for network simulation. Its formal structure and vendor support are strengths for foundational networking education. However, the programme is primarily defensive and networking-oriented, Packet Tracer does not support full operating system simulation or security tool integration, and the curriculum is tightly coupled to Cisco technologies.

E.4 SEED Labs

SEED Labs [7] is an open-source, research-backed collection of Docker-based security exercises covering specific concepts such as buffer overflows, SQL injection, and cryptographic protocols. The academic rigour and local deployability are significant advantages. Nevertheless, each lab is conceptually isolated, there is no structured penetration testing workflow spanning multiple systems, and significant instructor customisation is required to integrate SEED exercises into a complete curriculum.

E.5 Game of Active Directory (GOAD)

GOAD [8] provides an Infrastructure-as-Code vulnerable Active Directory environment designed for realistic Windows domain penetration testing. Its use of Terraform and Vagrant enables reproducible deployment and the scenarios reflect real corporate environments. The platform is, however, narrowly focused on Windows AD: it does not cover reconnaissance, web application vulnerabilities, or network traffic analysis, and it lacks formal instructor resources or curriculum structure.

E.6 VulnHub

VulnHub [9] is a community-driven repository of downloadable vulnerable virtual machines for local deployment. Its open, self-directed nature suits independent learners well, but the absence of structured progression, inconsistent quality across community-contributed machines, and lack of automation for deployment and reset make institutional integration impractical without significant instructor effort.

E.7 Hack4u Academy

Hack4u Academy [10] stands out for its emphasis on penetration testing methodology over tool memorisation, offering structured video-based courses with integrated practical exercises. Its curriculum is well-designed and the community active. The platform is, however, subscription-based and externally hosted, making local institutional deployment and automated large-scale assessment impossible.

F Objectives: Deliverables, Detailed KPIs, and Audience

This appendix complements Chapter 4 with the full deliverable specifications per specific objective, the detailed per-category KPI tables, and the extended audience analysis that frames the project's reuse potential.

F.1 Deliverables by Specific Objective

F.1.1 OE1: Design and Implement EVE-NG Labs

- 4 `.unl` topology files (Lab 1: implemented; Labs 2–4: planned)
- 8–12 optimised virtual machine images
- Technical configuration documentation for each lab

F.1.2 OE2: Develop Automation Scripts

- Bash/Python utility scripts for common tasks (e.g. bridge configuration, ISO upload, node connectivity checks)
- Scripts documented with usage instructions in the repository

F.1.3 OE3: Develop Structured Teaching Material

- Student lab guide PDF per lab — task description, objectives, hints, and tools reference
- Instructor resolution guide PDF per lab — full walkthrough, flag values, rubric (100 pts), and common errors
- Technical configuration reference per lab in the repository

F.1.4 OE4: Validate Usability and Educational Effectiveness

- Pilot session report with timing and usability observations
- Post-session survey results (satisfaction, difficulty, clarity)
- List of improvements implemented based on pilot feedback

Pilot note: The pilot group comprises 4–5 individuals from the GEISI student population, potentially including participants with documented learning difficulties (e.g. dyslexia), to assess the effectiveness of the accessibility measures incorporated in the lab documentation.

F.2 Detailed KPI Tables

The following tables break down the consolidated KPIs from Table 4.1 by category for reference.

F.2.1 Technical Indicators

Metric	Objective	Measurement Method
Lab deployment time	$\leq$ 2 min	Automated timing
VM boot success rate	$\geq$ 95%	System logs + validation scripts
Full reset time	$\leq$ 30 s	Scripts with timestamps
Scenario reproducibility	100%	Automated tests

Table F.1: Technical KPIs for EVE-NG lab deployment

F.2.2 Educational Indicators

Metric	Objective	Measurement Method
Lab resolution time	90–120 min	Time tracking during pilot
Lab completion (pilot)	All participants	Observation during pilot sessions
User satisfaction	$\geq$ 4/5	Post-use survey
Key concept understanding	$\geq$ 80% correct	Assessment questionnaire

Table F.2: Educational KPIs for lab effectiveness

F.2.3 Quality Indicators

Metric	Objective	Measurement Method
Documentation coverage	100% of features	Verification checklist
Critical errors	0	Exhaustive testing
EVE-NG version compatibility	100%	Tests in multiple environments
Interface usability	$\geq$ 4/5	UX heuristic evaluation

Table F.3: Quality KPIs for lab package

F.3 Extended Audience Analysis

F.3.1 Secondary End Users

- **Cybersecurity Faculty**: Other teachers interested in reusing the material for related courses
- **Postgraduate Students**: Possible extensions of the material for advanced levels

F.3.2 Potential Audience Beyond Tecnocampus

Internal Scope (Tecnocampus)

- Other courses related to cybersecurity

- Research projects in computer security
- Continuing education and professional certifications

External Scope

- Educational institutions with training programmes in cybersecurity
- Specialised vocational training centres
- Companies with internal security training programmes

G Methodology: Detailed Phase Activities and Quality Assurance

This appendix expands the phase summaries in Chapter 5 with full activity lists, quality assurance procedures, and documentation and version control practices.

G.1 Detailed Phase Activity Lists

G.1.1 Phase 0: Preparation and Infrastructure (July–November 2025)

- HomeLab infrastructure initialisation (Proxmox[19], networking, storage)
- Development environment setup (VSCode[25], LaTeX[26], Git)
- Security tools installation (Parrot OS Security[20], Metasploit[27], Burp Suite[28])
- Technical documentation and baseline establishment

Deliverables: Fully operational virtualisation infrastructure; development environment configuration; baseline documentation of system architecture.

Duration: 105 hours **Status**: Completed (100%)

G.1.2 Phase 1: Preliminary Project Proposal and Conceptual Design (October 2025–January 2026)

1. **SMART Objectives Definition** (4h) — Specification of Main Objective (MO): develop a reusable teaching package. Definition of 4 Specific Objectives (OE1–OE4) with measurable KPIs. Alignment with institutional GEISI curriculum requirements.

2. **State of the Art Analysis** (15h) — Comprehensive review of 7 major cybersecurity training platforms (HackTheBox, TryHackMe, Cisco Academy, SEED Labs, GOAD, VulnHub, Hack4u). Identification of pedagogical gaps. Comparative analysis matrix across multiple dimensions.

3. **Functional and Non-Functional Requirements** (10h) — RF1: design and implement 4 EVE-NG topologies (.unl files). RF2: implement vulnerabilities per PTES[29], OWASP[30], NIST[14], and CEH[3] standards. RF3: create structured assessment materials and rubrics. NFR1–NFR3 covering performance, framework compliance, and institutional sustainability.

4. **Pentesting Methodology and Validation Framework** (8h) — Adaptation of PTES for educational context. Definition of ethical hacking principles (EC-Council CEH). Mapping of attack phases to lab objectives. Validation criteria for security effectiveness.

5. **Design Decisions Justification** (5h) — Rationale for EVE-NG selection vs alternative platforms. Technology stack justification (Parrot OS Security, Metasploit, Burp, OWASP ZAP). Pedagogical approach: hands-on labs with automation.

6. **Risk Analysis and Mitigation Planning** (15h) — Identification of 12+ project risks (technical, pedagogical, institutional). Assessment of impact and probability. Definition of mitigation strategies.

7. **Resource Inventory and Allocation** (5h) — Hardware requirements: dual-socket server with Proxmox. Software stack: EVE-NG Community Edition, Parrot OS Security images, security tools. Human resources: student (~820h total), faculty supervisor. Budget considerations and open-source alternatives.

8. **Gantt Chart and Critical Path Analysis** (20h) — Development of detailed project timeline with 37 tasks. Identification of 13 critical path tasks. CSV export and interactive web-based visualisation. Progress tracking and milestone definition.

Deliverables: Formal project proposal (Preliminary Project Proposal); detailed Gantt chart with critical paths; risk register and mitigation

plan; requirements specification (functional and non-functional).

Duration: 82 hours **Status**: Completed **Milestone Deadline**: January 16, 2026

G.1.3 Phase 2: Implementation and Interim Validation (January–March 2026)

1. **Lab 1: Reconnaissance Development** (14h research + 20h dev) — Scenario: intelligence gathering phase using PTES framework. Tools: Nmap[34], passive reconnaissance techniques. Topology: multi-tier network with passive monitoring capabilities. Validation: successful execution of reconnaissance workflow.

2. **Lab 2: Web Vulnerabilities Development** (14h research + 25h dev) — Scenario: OWASP Top 10 vulnerability exploitation. Tools: Burp Suite, OWASP ZAP, manual testing techniques. Topology: web application server with vulnerable services. Validation: exploitation success and reporting accuracy.

3. **Lab 3: Incident Response Development** (14h research + 25h dev) — Scenario: log forensics and incident response (NIST SP 800-61[46] standards). Tools: Wireshark[35], tcpdump, network enumeration tools. Topology: multi-segment network with pre-staged attack artefacts. Validation: successful timeline reconstruction and flag recovery.

4. **Pilot Validation with Users** (20h + 6h revision) — Informal testing sessions with 4–5 GEISI students conducted as each lab is completed. Usability observations and timing notes. Post-session survey on satisfaction, difficulty, and guide clarity. Adjustments applied based on participant feedback.

5. **Topology Adjustments and Optimisation** (30h) — Performance tuning based on pilot feedback. VM image optimisation and deployment time reduction. Network configuration refinement for stability.

6. **Teaching Material Production** (40h) — Student lab guide PDF per lab: task description, objectives, hints, and tool reference. Instructor resolution guide PDF per lab: one possible walkthrough,

flag values, assessment rubric, and common errors. Technical configuration reference per lab in the repository.

Deliverables: 3 fully functional EVE-NG labs; student and instructor guides; pilot validation report; technical configuration references.

Duration: 188 hours **Status**: Completed **Milestone Deadline**: March 20, 2026

G.1.4 Phase 3: Completion and Final Validation (March–May 2026)

1. **Lab 4: Privilege Escalation Development** (16h research + 30h dev) — Scenario: post-exploitation and privilege escalation (CEH framework). Tools: Metasploit, manual exploitation, system enumeration. Topology: hardened systems with privilege escalation vectors. Validation: successful privilege elevation and system compromise.

2. **Comprehensive Penetration Testing** (30h Round 1 + 25h Round 2) — Round 1: sequential testing of Labs 1, 2, and 3. Round 2: integrated testing across all 4 laboratories. Documentation of all findings and remediation strategies.

3. **Automation Scripts Development** (30h) — Utility scripts (Bash/Python) for recurring lab management tasks: bridge configuration, ISO upload, and connectivity check scripts. Scripts documented and tested in the project EVE-NG environment.

4. **Final User Validation and Iteration** (18h) — Final informal validation sessions with the pilot group (4–5 participants). Collection of post-session survey data and usability notes. Implementation of final adjustments based on feedback.

5. **Bachelor's Thesis Report Writing and Finalisation** (90h) — Phase I (15h): Introduction, Objectives, Literature Review. Phase II (40h): Methodology, Requirements, Implementation Results. Phase III (35h): Validation, Analysis, Conclusions. Final revision, bibliography integration, and publication preparation (35h).

Deliverables: 4 fully functional labs; utility automation scripts; final bachelor's thesis report; pilot validation report; deployment package ready for institutional use.

Duration: 315 hours **Status:** In progress **Final Milestone Deadline:** May 27, 2026

G.2 Quality Assurance Procedures

G.2.1 Technical Validation

- Automated testing of lab deployment and reset
- Manual security testing and penetration verification
- Performance benchmarking against KPI targets
- Compatibility testing across EVE-NG versions

G.2.2 Pedagogical Validation

- Alignment with GEISI learning outcomes
- Assessment of student understanding through rubrics
- Effectiveness in achieving educational objectives
- Feedback from faculty and student participants

G.2.3 Usability Testing

- User interface and documentation clarity
- Accessibility and ease of deployment
- Support and documentation adequacy
- User satisfaction scoring ($\geq$ 4/5 target)

G.3 Documentation and Version Control

- GitHub repository (TheEgea/TFG) for all code, scripts, and documentation, licensed under CC BY-NC-SA 4.0
- LaTeX (XeLaTeX + OpenDyslexic) for formal academic documentation (Volume I and Volume II)
- MkDocs Material for the operational web documentation site
- Versioning system for all deliverables with semantic commit messages
- Change logs and improvement tracking through pull requests
- Deployment guides and runbooks per lab

H Requirements: Detailed Sub-Requirements

This appendix expands the requirements in Chapter 6 with the full sub-requirement specifications for each functional requirement (RF), technological requirement (TR), and non-functional requirement (NFR).

H.1 Functional Requirements: Sub-Specifications

H.1.1 RF1: Design and Implement EVE-NG Topologies

- **RF1.2**: Each topology must include:
  - Minimum 5–8 virtual machines per lab
  - Realistic network architecture (DMZ, internal networks, monitoring zones)
  - Vulnerable services configured per OWASP and NIST standards
  - Network segmentation with VLANs where appropriate
- **RF1.3**: EVE-NG compatibility requirements:
  - .unl file format compatible with EVE-NG Community Edition
  - Additional compatibility with EVE-NG Professional Edition
  - KVM hypervisor support (Ubuntu 20.04+ LTS)
  - Minimum host specification: 16 GB RAM, 200 GB storage
- **RF1.4**: Topology documentation:
  - Network diagram for each lab (logical and physical views)
  - IP addressing scheme documented
  - Service inventory and port mappings
  - Attack surface identification and justification

H.1.2 RF2: Implement Vulnerabilities and Attack Scenarios

- **RF2.5**: Vulnerability implementation standards:
  - All vulnerabilities must be fixable by administrators
  - No permanently broken or unrecoverable systems
  - Clear reset procedure for each vulnerability state
  - Automated scanning tools must successfully identify vulnerabilities

H.1.3 RF3: Create Structured Assessment Materials

- **RF3.3**: Support materials:
  - Student lab manuals with step-by-step guidance
  - Teacher guides with deployment and customisation instructions
  - Expected outcomes and common pitfalls documentation
  - Troubleshooting guides for common technical issues
- **RF3.4**: Validation materials:
  - Automated validation scripts to verify successful lab completion
  - Manual verification procedures for educators
  - Documentation of success indicators

H.2 Technological Requirements: Sub-Specifications

H.2.1 TR1: Virtualisation Infrastructure

- **TR1.2**: Hypervisor:
  - Primary: KVM (Kernel-based Virtual Machine)
  - Host OS: Ubuntu Server 20.04 LTS or 22.04 LTS
  - QEMU integration for VM management

  - Nested virtualisation support for advanced scenarios

- **TR1.4**: Network configuration:
  - Layer 2 switching capabilities within EVE-NG
  - VLAN support for network segmentation scenarios
  - IPv4 and IPv6 support

H.2.2 TR2: Security Tools and Penetration Testing Framework

- **TR2.4**: Compliance and documentation:
  - All tools must be freely available or have educational licences
  - Open-source preference for long-term sustainability
  - Tool version specifications documented
  - Legal and ethical use guidelines included

H.2.3 TR3: Scripting and Automation

- **TR3.3**: Integration points:
  - EVE-NG API integration for programmatic lab management
  - Webhook support for event-driven automation
  - Container orchestration (Docker) for microservices
  - Version control integration (Git, GitHub)

H.3 Non-Functional Requirements: Detailed Subsections

H.3.1 NFR1: Performance and Efficiency

NFR1.1 – Deployment Performance:

- Lab deployment time $\leq$ 2 minutes from topology load to ready state

- VM boot time $\leq$ 90 seconds per VM (95% of cases)
- Reset operation $\leq$ 30 seconds to restore to initial state
- Concurrent lab instances: support minimum 5 simultaneous labs

NFR1.2 – Resource Optimisation:

- VM image sizes $\leq$ 5 GB per image (compressed)
- Memory footprint $\leq$ 6 GB per lab instance during execution
- Minimal external dependencies (all local simulation)

NFR1.3 – Scalability:

- Current infrastructure supports 3–4 concurrent instances on the project HomeLab server
- Architecture horizontally scalable: additional EVE-NG nodes can be added
- EVE-NG Professional Edition supports larger concurrent deployments

H.3.2 NFR2: Compliance with Security Frameworks

NFR2.1 – NIST Cybersecurity Framework [14]: Core Functions (Identify, Protect, Detect, Respond, Recover) mapped to lab exercises; assessment procedures aligned with NIST guidelines.

NFR2.2 – CIS Critical Controls [13]: Controls 1–18 incorporated where pedagogically appropriate; assessment rubrics aligned with CIS benchmarks; remediation procedures follow CIS recommendations.

NFR2.3 – OWASP Standards [12], [30]: OWASP Top 10 fully covered; OWASP Testing Guide methodology integrated; development practices reflected in lab design.

NFR2.4 – Penetration Testing Standards [3], [29]: PTES phases mapped to labs; CEH curriculum alignment; ethical hacking principles strictly enforced.

H.3.3 NFR3: Reliability and Availability

NFR3.1 – System Reliability:

- MTBF $\geq$ 100 hours continuous operation
- MTTR $\leq$ 5 minutes
- Scenario reproducibility: 100% (deterministic behaviour)
- Zero critical bugs at release

NFR3.2 – Data Integrity: No unintended data loss during lab operations; snapshot and restore functionality; backup procedures for sensitive configurations.

NFR3.3 – Availability: Planned maintenance windows weekends only; system availability $\geq$ 99% during instructional periods; graceful degradation if resources become limited.

H.3.4 NFR4: Security and Ethical Compliance

NFR4.1 – Educational Sandbox Isolation: Complete network isolation from production systems; no outbound internet access from lab environments; contained attack scenarios (no external targets).

NFR4.2 – Access Control and Accountability: Role-based access control (RBAC) for lab management; audit logs of all student actions where applicable; clear identification of authorised vs. unauthorised activities.

NFR4.3 – Ethical Guardrails: Mandatory ethics training before lab access; written agreements regarding responsible use; monitoring and escalation procedures for suspicious activities.

H.3.5 NFR5: Usability and Support

NFR5.1 – Ease of Use: Average faculty deploy time 5–10 minutes; intuitive web interface with clear navigation; minimal technical prerequisites for students.

NFR5.2 – Documentation and Support: Comprehensive user manuals and quick-start guides; FAQ documentation; responsive support channel.

NFR5.3 – Accessibility: WCAG 2.1 Level AA compliance for web interfaces; screen reader compatibility; keyboard navigation support; clear contrast ratios.

H.3.6 NFR6: Institutional Sustainability

NFR6.1 – Long-term Maintainability: All components open-source or freely available; no vendor lock-in; source code in GitHub with clear documentation; backward compatibility with older EVE-NG versions where feasible.

NFR6.2 – Curriculum Alignment: Explicit mapping to GEISI degree learning outcomes; integration with existing course structures; flexible customisation for different instructional contexts.

NFR6.3 – Scalability for Institutional Adoption: Deployable on institutional infrastructure (not SaaS-dependent); support for multiple faculty deployments simultaneously; resource reservation and scheduling capabilities.

I Feasibility: Task Breakdown and NIST Risk Assessment

This appendix complements Chapter 3 with the full per-phase task lists (task IDs, estimated hours, critical path flags) and the five-phase NIST SP 800-30 risk assessment that underpins the risk matrix in that chapter.

I.1 Phase Task Lists

I.1.1 Phase 0: Preparation (July–November 2025)

- Task 0.1: HomeLab Initialisation Setup (64h)
- Task 0.2: VSCode Setup for thesis development (18h)
- Task 0.3: Parrot OS Security Setup and Basic Tools (15h)
- Task 0.4: Home Lab As-Built Documentation (8h)

Duration: 105 hours **Status**: Completed (100%)

I.1.2 Phase 1: Preliminary Project Proposal (October 2025–January 2026)

- Task 1.0: Thesis Development (22h)
- Task 1.1: SMART Objectives Definition (4h)
- Task 1.2: State of the Art: Frameworks and Tools (15h)
- Task 1.3: Functional and Non-Functional Requirements (10h)
- Task 1.4: Pentesting Methodology and Validation (8h) — **CRITICAL**
- Task 1.5: Design Decisions and Justification (5h)
- Task 1.6: Gantt Chart Development (20h)

- Task 1.7: Risk Analysis and Mitigation Plan (15h)
- Task 1.8: Resource Inventory (5h)
- **Milestone 1.10**: Preliminary Project (40h) — **Delivery: January 16, 2026**

Duration: 82 hours **Status**: Completed

I.1.3 Phase 2: Interim Report (January–March 2026)

- Task 2.1: Lab 1 Research — Reconnaissance (14h) — **CRITICAL**
- Task 2.2: Lab 2 Research — Web Vulnerabilities (14h) — **CRITICAL**
- Task 2.3: Lab 3 Research — Network Analysis/IR (14h) — **CRITICAL**
- Task 2.4: Technical Write-ups Labs 1 & 2 (13h)
- Task 2.5: Topology Adjustments (30h)
- Task 2.6: Pre-validation with Pilot Users (20h) — **CRITICAL**
- Task 2.6.1: Pre-validation Revision (6h) — **CRITICAL**
- Task 2.7: Analyse and Write Hackathon Results (17h)
- Task 2.8: Interim Report Formalisation (40h)
- Task 2.9: Set Up Servers, VMs, and EVE-NG (20h)
- Task 2.10: Backup and Documentation of EVE-NG (10h)
- **Milestone 2.11**: Intermediate Thesis (50h) — **Delivery: March 20, 2026**

Duration: 212 hours **Status**: Completed

I.1.4 Phase 3: Final Delivery (March–May 2026)

- Task 3.1: Lab 4 Research — Post-Exploitation (16h) — **CRITICAL**
- Task 3.2: Lab 3 Validation with Users (12h)
- Task 3.3: Technical Write-ups Labs 3 & 4 (19h)
- Task 3.4: Penetration Testing Round 1 (Labs 1–2–3) (30h) — **CRITICAL**
- Task 3.5: Penetration Testing Round 2 (All Labs) (25h) — **CRITICAL**
- Task 3.6: Python Automation Scripts (30h)
- Task 3.7: Final Validation with Pilot Users (18h)
- Task 3.8: Final Review and Project Polishing (25h)
- Task 3.9: Bachelor's Thesis Report Writing I (15h) — **CRITICAL**
- Task 3.10: Bachelor's Thesis Report Writing II (40h) — **CRITICAL**
- Task 3.11: Bachelor's Thesis Report Writing III (35h) — **CRITICAL**
- Task 3.12: Final Revision (15h)
- Task 3.13: Final Report Preparation (20h)
- **Milestone 3.14**: Final Report (5h) — **Delivery: May 27, 2026**

Duration: 315 hours **Status**: In progress

I.2 NIST SP 800-30 Five-Phase Risk Assessment

This section details the systematic risk assessment following NIST SP 800-30[21]. The summary risk matrix is provided in Table 3.7 in Chapter 3.

I.2.1 Phase 1: Asset Identification and Scope Definition

Critical assets are categorised by CIA triad:

Hardware Assets:

- HomeLab server (2× Xeon E5-2680v4, 64 GB RAM) — Availability critical
- Development workstations (laptop + desktop) — Integrity critical
- Network equipment (router, switch) — Availability critical
- External storage (backup) — Confidentiality + Availability critical

Software Assets:

- EVE-NG Community Edition — Availability critical
- Parrot OS Security distribution — Integrity critical
- Vulnerable VMs (Metasploitable, DVWA) — Integrity critical
- Development tools (VSCode, Git, LaTeX) — Integrity critical

Data Assets:

- Thesis source code and documentation — Confidentiality + Integrity critical
- Lab topology configurations — Integrity + Availability critical
- Student learning materials — Availability critical
- Project research findings — Confidentiality + Integrity critical

Scope: Infrastructure and technical risks within the controlled laboratory environment. Excluded: organisational, regulatory, and external threats beyond the academic context.

I.2.2 Phase 2: Threat and Vulnerability Identification

Threat categories (NIST SP 800-30):

- **Hardware Threats:** Disk failures, power supply degradation, CPU overheating, memory exhaustion
- **Software/Configuration Threats:** Outdated VM images, incompatible versions, configuration drift, EVE-NG crashes
- **Knowledge Gap Threats:** Insufficient expertise in advanced networking, virtualisation optimisation, security best practices
- **Resource Constraint Threats:** Limited RAM for simultaneous VM operations, storage capacity limits
- **Integration Threats:** EVE-NG network compatibility issues, tool interoperability
- **Data Loss Threats:** Accidental deletion, corruption, hardware failure

Key vulnerabilities:

- Aging server components (end-of-life approaching)
- Single point of failure — no infrastructure redundancy
- EVE-NG Community Edition has no commercial support
- Limited RAM capacity for production-scale virtualisation
- Inadequate documentation of network topology state
- Limited backup redundancy (single backup location)

I.2.3 Phase 3: Impact and Likelihood Analysis

Scales follow NIST SP 800-30: Probability (Low $<$25%, Medium 25–75%, High $>$75%); Impact (Low: minor delays, Medium: project delays, High: project jeopardy).

Risk Scenario	Prob.	Impact	Level	Primary Asset
EVE-NG Performance Issues	Medium	High	HIGH	Infrastructure, Labs
VM Image Incompatibility	Medium	Medium	MEDIUM	Development, Labs
Knowledge Gaps (advanced net.)	Medium	Medium	MEDIUM	Development, Timeline
Hardware Degradation / Disk Failure	Low	High	MEDIUM	Infrastructure
Data Loss (insufficient backup)	Low	High	MEDIUM	Documentation
Configuration Documentation Gaps	Medium	Medium	MEDIUM	Operations

Table I.1: Risk assessment matrix with probability and impact (NIST SP 800-30)

I.2.4 Phase 4: Risk Evaluation and Prioritisation

HIGH Priority: EVE-NG Performance Issues — infrastructure bottlenecks causing lab functionality failures. Medium probability, high impact. Requires immediate resource optimisation and monitoring implementation.

MEDIUM Priority:

1. VM Incompatibility — disparate image formats causing integration problems
2. Knowledge Gaps — experience gaps in advanced networking/virtualisation
3. Configuration Documentation — inadequate topology documentation
4. Data Loss — insufficient backup redundancy (single point of failure)

LOW Priority: Hardware Degradation — aging components manageable through preventive maintenance.

I.2.5 Phase 5: Mitigation Strategies

EVE-NG Performance Issues (REDUCTION):

- Hardware dimensioning: 64 GB RAM dedicated to the server running EVE-NG (32 GB proved insufficient — additional module acquired, ~80€)
- Resource optimisation: memory tuning, VM suspension when idle
- Documented fallback: VirtualBox as alternative hypervisor if critical failure
- *Result*: Probability reduced from Medium to Low

VM Incompatibility (PREVENTION):

- Early testing: validate all images in target environment before use
- Format standardisation: QCOW2 for EVE-NG compatibility
- Alternative repositories maintained (VulnHub, GitHub)
- *Result*: Probability reduced from Medium to Low

Knowledge Gaps (PREVENTION + REDUCTION):

- Structured learning: Task 1.2 (15h) comprehensive framework research
- Continuous upskilling during lab research phases (Tasks 2.1–3.1)
- Expert consultation with tutor Pere Vidiella i Catalan
- Technical references: NIST[31], OWASP[30]
- *Result*: Impact reduced from Medium to Low-Medium

Configuration Documentation (PREVENTION):

- Continuous documentation: update topology diagrams during development
- Automated export: EVE-NG built-in topology export
- Version control: all network configs via Git repository

- *Result*: Probability reduced from Medium to Low

Data Loss (REDUCTION):

- Daily automated encrypted backups to external storage
- Version control: full Git history on GitHub
- Off-site cloud backup (student account)
- *Result*: Probability Low, Impact Low

Hardware Degradation (PREVENTION):

- Quarterly hardware diagnostics (disk health, temperature monitoring)
- Automated S.M.A.R.T. alerts for disk errors and CPU temperature thresholds
- Component procurement plan with 6-month horizon for critical parts

Risk Assessment Conclusion: All identified risks have been systematically evaluated and assigned concrete mitigation strategies. No individual risk is considered project-blocking.

I.3 Detailed Budget Tables

I.3.1 Software and Licences

Resource	Licence Type	Unit	Total
EVE-NG Community Edition	Free/Open Source	0€	0€
Parrot OS Security	Free/Open Source	0€	0€
Proxmox VE	Free/Open Source	0€	0€
Metasploitable VMs	Free/Open Source	0€	0€
DVWA	Free/Open Source	0€	0€
Visual Studio Code	Free/Open Source	0€	0€
LaTeX (TeX Live)	Free/Open Source	0€	0€
Git + GitHub (Student)	Free/Student	0€	0€
Burp Suite Community	Free	0€	0€
Wireshark	Free/Open Source	0€	0€
SUBTOTAL Software			**0€**

Table I.2: Software and licence costs (all free/open-source for educational use)

I.3.2 Other Operational Costs

Concept	Cost
Electricity (820 h × 0.15 kWh × 0.20 €/kWh)	25€
Internet connectivity (9 months × 40 €/month)	360€
Documentation printing and binding (3 copies)	60€
Contingency fund (5% of total)	635€
TOTAL Other Costs	**1,080€**

Table I.3: Detailed breakdown of additional operational costs

Note: The total budget summary in Chapter 3 (Table 3.5) uses a rounded figure of 1,020 €for other costs, reflecting the exclusion of the contingency fund from the base estimate.